



Population diversity and inheritance in genetic programming for symbolic regression

Bogdan Burlacu¹ · Kaifeng Yang¹ · Michael Affenzeller¹

Accepted: 10 November 2022 / Published online: 17 January 2023
© The Author(s) 2023. This article is published with open access at Springerlink.com

Abstract

In this work we aim to empirically characterize two important dynamical aspects of GP search: the evolution of diversity and the propagation of inheritance patterns. Diversity is calculated at the genotypic and phenotypic levels using efficient similarity metrics. Inheritance information is obtained via a full genealogical record of evolution as a directed acyclic graph and a set of methods for extracting relevant patterns. Advances in processing power enable our approach to handle previously infeasible graph sizes of millions of arcs and vertices. To enable a more comprehensive analysis we employ three closely-related but different evolutionary models: canonical GP, offspring selection and age-layered population structure. Our analysis reveals that a relatively small number of ancestors are responsible for producing the majority of descendants in later generations, leading to diversity loss. We show empirically across a selection of five benchmark problems that each configuration is characterized by different rates of diversity loss and different inheritance patterns, in support of the idea that each new problem may require a unique approach to solve optimally.

Keywords Genetic programming · Symbolic regression · Supervised learning · Explainable AI · Optimization algorithm · Evolutionary algorithm

1 Introduction

Genetic programming (GP) is an evolutionary meta-heuristic that performs a guided, stochastic search over a space of computer programs: syntactical constructs that encode a domain-specific way to solve a given problem.

The idea of allowing the computer to evolve programs itself can be traced back to the works of Turing (1950), Samuel (1959) or von Neumann (1966) in the 1950s and saw its first applications in the 1980s by Forsyth (1981), Cramer (1985) and Hicklin (1986). Later, genetic programming became popular and well known to the public thanks to John Koza's contributions Koza (1990, 1992) in

the 1990s. Each of these developments have been inspired by the earlier works by Holland (1975) and De Jong (1975a) in 1975.

Similar to other Evolutionary Algorithms (EAs), GP evolves a population of solution candidates by following the Darwinian principles of evolution (i.e., the survival of the fittest) and utilizing biologically-inspired operators (e.g., crossover, mutation, and selection). The feature that distinguishes GP from other EAs (e.g., genetic algorithm, evolution strategies) is its variable-length solution representation as S-expressions,¹ which inherently enables it to search for free-form structures using observed data without any domain knowledge.

The versatility and generality of S-expressions and the robustness of population-based evolutionary algorithms make GP particularly well-suited for solving regression problems by searching over the space of mathematical expressions. The application of GP for regression is known as symbolic regression (SR) (Tackett 1995). In SR, the

✉ Bogdan Burlacu
Bogdan.Burlacu@fh-hagenberg.at

Kaifeng Yang
Kaifeng.Yang@fh-hagenberg.at

Michael Affenzeller
Michael.Affenzeller@fh-hagenberg.at

¹ HEAL, University of Applied Sciences Upper Austria, Softwarepark 11, 4232 Hagenberg, Austria

¹ S-expressions or symbolic expressions are a general tree-structured (nested list) data representation popularized by the LISP programming language.

algorithm can evolve mathematical expressions with the goal of fitting an unknown function and return the best expression from the population as a result. Compared to black-box regression methods, this approach has the advantage of producing an understandable model from a human's perspective and is therefore useful in application areas such as system identification.

1.1 Motivation

As a general method that requires no a-priori knowledge, GP has several degrees of freedom which determine its search behavior. Parameters such as population size, generational limit, selection pressure, genetic operator probabilities, S-expression depth and length limits or primitive set all contribute to the success of the search. Finding appropriate values for these settings is a difficult process and furthermore these settings cannot be easily generalized across different problems (Smit and Eiben 2009). Ideally, an optimal parameterization is characterized by an appropriate balance between both the exploratory and exploitative aspects of the search, but different problems require different strategies for exploration and exploitation (Eiben and Schippers 1998; Črepinšek et al. 2013; Michalewicz 1996). Parameter tuning approaches range from trial and error to following general guidelines and performing hyperparameter optimization (De Jong 2007; Michalewicz and Schmidt 2007; Eiben et al. 2007; Lobo and Lima 2007; Meyer-Nieberg and Beyer 2007).

In the context of GP, exploratory behavior enables recombination operators to sample different regions of the search space while exploitative behavior enables the gradual, incremental refinement of existing solution candidates (Črepinšek et al. 2013). An ideal search typically starts with a short exploratory phase in the initial stages followed by a longer exploitative phase, increasing in intensity as the algorithm converges. The transition between the two phases takes place gradually under the effects of selection pressure as successful individuals multiply their own genes in the population while unsuccessful ones become extinct.

Since surviving genes propagate through inheritance, the study of inheritance patterns in GP can provide valuable insight into the effectiveness of the genetic search, the loss of diversity and occurrence of repeated patterns. This information can potentially help devise better strategies for the parameterization or automatic tuning of the algorithm.

Furthermore, more recently proposed evolutionary models such as offspring selection (OSGP) (Affenzeller et al. 2005) and age-layered population structure (ALPS) (Hornby 2006) have been specifically designed to alleviate the issue of diversity loss in GP populations and

should be considered as alternatives to the canonical algorithm as originally defined by Koza.

OSGP introduces a secondary selection step called offspring selection which enforces the rule that a generated child must be at least to some degree better than its parents in order to be accepted into the population. This ensures that only *adaptive* changes (e.g. mutations and crossovers that increase fitness) are accepted, thus encouraging fit alleles to surface earlier in the search. Its design makes OSGP less sensitive to the maximum number of generations parameter, since the search can terminate automatically when offspring that outperform their parents can no longer be produced.

ALPS assigns individuals an age value that is incremented with each function evaluation. This value is then used to divide the population into different age layers (or groups) organized hierarchically and evolved independently. The layers represent non-overlapping age intervals ordered from youngest to oldest. The interval size is defined as a function of the *age gap* parameter. Offspring inherit their parents' ages after genetic operators. When an individual's age is older than the maximum age of layer t , $t \in \{0, 1, \dots\}$, this individual will be migrated to layer $t + 1$ of higher maximum age in the hierarchy. Randomly generated, fresh individuals are constantly introduced at the bottom of the hierarchy (into the youngest age layer) to ensure a constant trickle of new genes into the population at the bottom layer. The algorithm nurtures these individuals by restricting the competition for survival only in the same age layer. Through this concept, ALPS aims to, on the one hand, promote population diversity and on the other hand, to achieve the idea of open-ended evolution, e.g. an evolutionary process which, through a constant influx of new genes, avoids premature convergence and can evolve indefinitely.

This empirical study of different evolutionary models on a collection of regression benchmark problems aims to highlight the different behaviors caused by different approaches to diversity preservation. Diversity is considered the most important dynamical aspect of GP. This knowledge can, on the one hand, help guide researchers of evolutionary algorithm design by informing on the expected effects of certain kinds of algorithmic extensions (e.g. age-, similarity- or genealogy-based), and on the one hand, it can help practitioners decide which approach is best suited for a particular problem and application domain.

The idea to identify and analyze patterns in GP populations is deeply related to properties such as diversity and convergence and has been thoroughly investigated both theoretically and empirically since the early 1990s. In the following we summarize the relevant approaches.

1.2 Schema-theoretic and empirical approaches

A schema is a genetic template or pattern that contains wildcard symbols and can match multiple individuals in the population. A schema theorem is a mathematical model for predicting the evolution of schema frequencies, where frequency is simply defined as the proportion of matching individuals. Originally formulated by John Holland for the genetic algorithm, the GA Schema Theorem offers an elegant probabilistic explanation for the propagation of fitter than average genetic patterns (schemata) within populations of binary strings (where schema fitness is the average fitness of matching individuals). Since the binary strings are all of the same length, the equations for conditional probabilities can be easily derived by taking into account the probabilities of the genetic operators themselves. The GA Schema Theorem helped establish genetic algorithms in the field of metaheuristics by proving that the evolutionary search converges and is superior to random search.

However, the mathematical models for the propagation of schemata during the evolutionary run become significantly more complex when dealing with variable-length S-expressions. The additional complexity introduced by variable-length encodings is a hard limiting factor for the scope and explanation power of schema theorems for GP.

Thanks to the work of Poli and collaborators, several schema theorems for GP have found applications in the study of bloat, operator behavior, population initialization or even parameterization of GP (Poli and McPhee 2009, 2009; Poli et al. 2010). However, the proposed mathematical models are large, hard to handle computationally and consequently hard to put into practice. These limitations have instead led to a plethora of empirical analysis methods that are intrinsically connected to the notions of population diversity and algorithm convergence. These methods attempt to record information about the search process and investigate whether it confirms the effects predicted by schema theorems.

Pattern mining approaches use frequency counting and tree matching algorithms to analyze changes in the genetic make-up of the population and measure loss of diversity. Overall they confirm schema-theoretical assumptions, for example that selection increases the frequency of fitter-than-average schemata (Poli and Langdon 1997; Majeed 2005; Smart et al. 2007) while causing the occurrence of large amounts of repeated patterns (semantic and

syntactic) (Wilson and Heywood 2005; Ciesielski and Li 2007; Langdon and Banzhaf 2008; Kameya et al. 2008; Joó 2010).

Genealogy analysis approaches aggregate information about lineages and hereditary relationships to investigate aspects of inheritance, diversity or operator behavior (e.g. the ratio of ancestors to descendants, the fitness of offspring compared to their parents). They find that few individuals from the initial population have surviving descendants in the last generation and that only a small fraction of ancestors contribute genes to the best solution (Donatucci et al. 2014; Burlacu et al. 2015a, b; McPhee et al. 2016, 2017). Additionally, network theory approaches indicate that algorithm convergence is preceded by the formation of large connected components within the genealogy graph (Kuber et al. 2014). The cited studies suggest that “modules” or “building blocks” emerge as a result of recombination and selection, and that in general, the choice of operators (selection in particular) has a big impact on search performance.

1.3 Genealogy analysis

In this approach, the genealogy of the best individual (or alternatively, of the entire population in the last generation) is analysed using notions of graph theory and GP-specific heuristics (e.g., graph traversal methods that distinguish between crossover root and non-root parents, are able to handle mutations or consider elitism). The genealogy is typically stored as a directed acyclic graph where vertices represent individuals and arcs represent hereditary relationships between them. The graph structure also stores relevant metadata such as fitness values or types of genetic operators that were applied and the coordinates of genotype changes. Graph traversal mechanisms are integrated with the approach in order to aggregate the necessary information, typically by performing a bottom-up traversal (i.e. from descendant to ancestor). Dynamical aspects such as the evolution of diversity over time are then able to be correlated with various quantifiable properties of the genealogy graph.

Kuber et al. (2014) used graph theory to analyze the properties of “ancestral networks” trying to identify large connected components as a sign of algorithm convergence, on the assumption that “the natural behavior of most species is to form cohesive communities (networks)”. Based on this information, they introduced a reseeding heuristic

that reinitialized a fraction of the population if the size of the connected component exceeded an arbitrary limit, with the end goal of tuning the balance between exploration and exploitation during the run.

Donatucci et al. (2014) recorded GP genealogies using graph database software and found that the percentage of initial individuals with direct descendants in the final generation is extremely small. They also observed that success rates of crossover and mutation are quite small (dropping to about 2% for mutation in the final generations).

Burlacu et al. (2015a, 2015b) recorded the genealogy of the entire population together with contextual information about the positions of subtrees swapped by crossover. This enabled the identification of the most frequently sampled subtrees in the entire genealogy, by working backwards from the final population. They find that a) most swaps done by crossover are ineffective as diversity decreases, b) a very small percentage of ancestors contribute genes to the final generation descendants and c) the most sampled subtrees are correlated to the problem building blocks.

McPhee et al. (2016, 2017) used graph databases to record genealogical information for entire runs using PushGP, a GP variant for software synthesis. They explored full genealogical records for 100 runs by working backwards from the last generation, and found that genealogy information can unveil significant new details about search behavior, particularly about the impact of selection. Algorithmic runs (and corresponding genealogies) can differ significantly between different selection methods (lexicase and tournament selection), significantly influencing convergence behavior. The main drawback of this approach is that the generated graph database is very large (e.g., a tournament selection dataset with 100 runs is 31 GB).

Genealogy analysis can help discover specific details about the process of evolution, such as patterns of inheritance (e.g., origins of useful genes in the ancestor population, which genes act in concert with other genes to form fit subtrees or building blocks, the mechanisms by which they are inherited), common ancestries (e.g., degree of relatedness between solutions), or the propagation of schemata.

1.4 Population diversity

The importance of population diversity has been widely recognized within the GP community (Michalewicz 1996),

as noted by McPhee and Hopper “progress in evolution depends fundamentally on the existence of variation in the population” (McPhee and Hopper 1999). Diversity refers to the differences between individuals, which can be quantified at the structural (genotype) level, at the semantic (phenotype) level or as a mixture of the two.

Some studies focused on genotype diversity counted frequencies of genes (De Jong 1975b; D’haeseleer and Bluming 1994) or different genotypes (Langdon 1998). McPhee and Hopper proposed three different criteria by contrasting a population with its ancestry population (e.g., the initial population). They concluded that most individuals from the initial population do not contribute any genetic material to the final population by using a canonical GP subtree crossover with no mutation operator (McPhee and Hopper 1999). Burke extended this idea by computing a ratio of unique subtrees to total subtrees (Burke et al. 2002). Other approaches to quantify genotype diversity include distance-based methods, using the Hamming metric (Eshelman and Schaffer 1993; Shimodaira 1997), Euclidean metric (Mc Ginley et al. 2011; Bessaou et al. 2000), edit distance (de Jong et al. 2001; Ekárt and Németh 2000)), structural distance through an information hypertree (iTree) (Ekárt and Gustafson 2004), bottom-up distance (Affenzeller et al. 2017) or entropy-based methods (Li et al. 2004; Burke et al. 2004; Misiūčiūš 2011). The problem of computing the edit distance between unordered labeled fixed-degree $k \geq 2$ trees has been shown to be NP-hard for optimization problems (Zhang et al. 1992; Bille 2005).

One point of criticism for genotype diversity is that a genetically diverse population does not guarantee behavioral diversity, since many trees can evaluate to the same expression or behavior (Burks and Punch 2018). This is due to the complex and non-injective genotype-to-phenotype mapping typical of GP. Instead, phenotype diversity was suggested to better serve the purpose of ensuring diverse behaviors in the population. Some approaches to quantify phenotype diversity consist of calculating ranks of individuals (Luerssen 2005), computing the Hamming distance-to-average-point (Ursem 2002), entropy-based methods (Rosca 1995; Burke et al. 2004) or correlation distance (Neshatian and Zhang 2009; Shirkhorshidi et al. 2015; Winkler et al. 2018; Agapitos et al. 2019).

Many different approaches for maintaining diversity at either the genotype or phenotype levels, or more effectively exploiting remaining diversity have been studied (Jackson 2010; Črepinšek et al. 2013; Burke et al.

2004; Burks and Punch 2018). Promoting diversity as a secondary goal has also been shown to work very well in the multi-objective case (de Jong et al. 2001; Burlacu et al. 2019).

In this paper we focus on diversity at both genotype and phenotype levels. We note cases where the studied algorithms behave differently in terms of diversity and correlate these behaviors with observed search outcomes. Diversity is computed using appropriate distance measures based on tree isomorphism and Pearson's correlation for genotypes and phenotypes, respectively. The remainder of this work is structured as follows: Sect. 2 introduces the algorithms that are tested in this paper. Section 3 defines the dynamic analysis methodology. Section 4 describes the test problems and parameter settings of algorithms, and shows experimental results. Section 5 concludes with a summary and discusses future work.

2 Algorithm description

In this section, the algorithmic variants introduced in Sect. 1.1 – OSGP and ALPS – are described in technical detail. Both algorithms are seen as extensions to canonical GP which introduce specific augmentations (i.e. an additional selection step for OSGP, age-based *niching* for ALPS) in order to improve population diversity, as well as the overall algorithm's ability to exploit existing diversity and generate adaptive change. Our analysis will then consider these augmentations and quantify their effects on evolutionary dynamics, from the perspective of diversity (measured at the genotype and phenotype level) and properties of the resulting genealogy graphs.

2.1 Genetic programming

Genetic programming is capable of solving optimization problems which can be formulated as:

$$\arg \min f(\mathbf{x}), \quad \mathbf{x} \in \mathcal{X} \quad (2.1)$$

where $\mathbf{x} = (x_1, \dots, x_l)^\top$ represents a decision vector (also known as individual) in EAs, $\mathcal{X}$ is the search space for the decision vector $\mathbf{x}$, and $f(\cdot)$ is a function to measure how good/bad a decision vector $\mathbf{x}$ is.

Unlike the canonical Genetic Algorithm (Holland 1975) and Evolution Strategies (Rechenberg 1965), the search space $\mathcal{X}$ in GP is not merely a binary space $\mathbb{D}$ or a

continuous space $\mathbb{R}$, but a union set of a function space $\mathcal{F}$ and a terminal node space $\mathcal{TN} = \{s, c\}$, where $s \in \mathcal{S}$ represents a symbol and $c \in \mathbb{R}$ represents a constant number. The function set $\mathcal{F}$ includes available symbolic functions (e.g., $\mathcal{F} = \{+, -, \times, \div, \exp, \log\}$) which require at least one terminal node as an argument. A terminal node in $\mathcal{TN}$ can be either a symbol variable x or a constant number c . Therefore, compared with GA and ES, the structure of a GP individual (i.e., relationships among variables) can be evolved and optimized. To emphasize the ability of structure evolution, a GP individual is denoted as T in this paper, where $T = (t_1, \dots, t_l)^\top$ and $t_i \in \{\mathcal{F} \cup \mathcal{TN}\}_{i \in \{1, \dots, l\}}$ is a node in the tree encoding.

Many other encoding strategies may be used to compose a meaningful symbolic expression such as linear encoding (Holmes and Barclay 1996), graph-based encoding (Poli and Langdon 1998), a grammar-based encoding (McKay et al. 2010) and others. However, the canonical tree encoding (Koza 1992) remains the most popular due to its simplicity (allowing for straightforward of genetic operators) and is used in this paper.

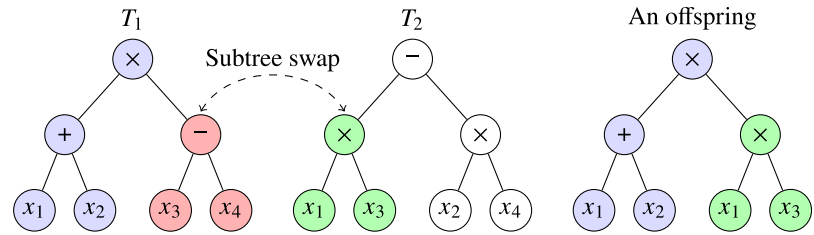
In a syntax tree, an *internal node* represents a mathematical function or logic operator in $\mathcal{F}$ and a *leaf node* is a terminal such as a variable or constant in $\mathcal{TN}$. An individual $T = (t_1, \dots, t_l)^\top$ is an ordered sequence of function symbols and terminals, where the first element represents a function in $\mathcal{F}$ and the following elements represent arguments for the first element. Some other related terminologies are introduced here:

1. A *root node* is the topmost node of a tree.
2. A *tree depth* (d) is the number of edges on the longest path from a root to a leaf.
3. A *tree length* (l) is the total number of nodes (including the root node) contained in the tree.

Similar to other EAs, three basic operators are applied in GP crossover, mutation, and selection. The pseudocode of GP is shown in Algorithm 1. Our GP implementation employs elitist selection, where the best individual of the current generation is automatically transferred to the next generation. We use the notation $B(p)$ to denote a single-outcome Bernoulli trial:

$$B(p) = \begin{cases} 1 & \text{with probability } p \\ 0 & \text{with probability } 1 - p \end{cases}$$

Fig. 1 An example of the *swapping tree crossover* in GP



Algorithm 1: Pseudocode of canonical genetic programming

Input: Objective function $f(\cdot)$, Population size n_{pop} , crossover rate p_c , mutation rate p_m , number of nodes l , depth d ;
Output: optimal solution T^*

```

1  $t = 0$ ;
2  $Pop(t) \leftarrow \text{InitializePopulation}(n_{pop}, d, l)$ ;
3  $\text{EvaluatePopulation}(Pop(t))$ ;
4 while not terminate do
5    $Pop(t+1, 0) = \arg \min f(T)$ , where  $T \in Pop(t)$ ;
6    $i = 1$ ;
7   while  $i < n_{pop}$  do
8     if  $B(p_c) == 1$  then
9        $P_1, P_2 \leftarrow \text{MateSelection}(Pop(t))$ ;
10       $Pop(t+1, i) \leftarrow \text{Crossover}(P_1, P_2)$ ;
11      if  $B(p_m) == 1$  then
12         $Pop(t+1, i) \leftarrow \text{Mutation}(P_1)$ ;
13      else if  $B(p_m) == 1$  then
14         $P_1 \leftarrow \text{MateSelection}(Pop(t))$ ;
15         $Pop(t+1, i) \leftarrow \text{Mutation}(P_1)$ ;
16       $i = i + 1$ ;
17  $\text{EvaluatePopulation}(Pop(t+1))$ ;
18  $T^* \leftarrow \arg \min f(T)$  where  $T \in Pop(t+1)$ ;
19  $t = t + 1$ ;

```

Crossover is an operator that generates offspring by swapping parts of the parents' genotypes and is an important exploration technique in EAs. In a tree-encoded individual, the most common crossover is the *subtree swapping crossover*, where two nodes t' and t'' are randomly sampled from two parents T_1 and T_2 , respectively. Then the subtrees T' and T'' , with root nodes t' and t'' , are exchanged between parents T_1 and T_2 . Typically, as suggested by Koza in Koza (1992), the crossover operator performs a biased selection of nodes to be swapped where internal (function) nodes are prioritized, as swapping leaf nodes leads to less meaningful results.

Example 1 Suppose two parents $T_1 = (\times, +, x_1, x_2, -, x_3, x_4)^\top$ and $T_2 = (-, \times, x_1, x_3, \times, x_2, x_4)^\top$, two random nodes are $t' = \{-\}$ and $t'' = \{\times\}$ for T_1 and T_2 , respectively. Then, one of the offspring generated by the *subtree swapping crossover* for T_1 and T_2 can be $(\times, +, x_1, x_2, \times, x_1, x_3)^\top$, as shown in Fig. 1.

Mutation in EA acts mostly as a provider of genetic diversity to work against premature convergence. We allow mutation to perform random changes to the tree structure,

such as removing subtrees, replacing subtrees, or altering node types or node parameters.

In GP problems, an objective function quantitatively measures the difference between predictions and actual responses on a specific dataset. For this purpose, some common objective functions for supervised learning problems are the *mean squared error*, *normalized mean squared error*, *coefficient of determination*, or the *Pearson correlation coefficient*. In this paper, the *Pearson correlation coefficient squared*² is used to measure a model's prediction error.

2.2 Offspring selection with GP

Offspring selection GP Affenzeller et al. (2005) conditionally accepts a new offspring into the population based on its fitness. The pseudocode of general offspring selection, which relates to the loop between line 7 and 14 in Algorithm 1, is shown in Algorithm 2.

² As this paper is restricted to minimization problems, we can switch the optimizing the *Pearson correlation coefficient squared* from a maximization problem into a minimization problem by simply adding a minus symbol.

Algorithm 2: Pseudocode of GP with offspring selection

Input: population $Pop(t)$, success ratio r , comparison factor c , selection pressure p_{sel} , objective function $f(\cdot)$, population size n_{pop} , crossover rate p_c , mutation rate p_m , discarded population pool $POOL$;

Output: Offspring population $Pop(t+1)$

```

1  $i = 0$ ;
2  $Pop(t+1) = \emptyset$ ;
3 while  $i < r \cdot n_{pop}$  and  $p_{active} < p_{sel}$  do
4   if  $B(p_c) \cdot B(p_m) = 1$  then
5      $P_1, P_2 \leftarrow \text{MateSelection}(Pop(t))$ ;
6      $child \leftarrow \text{Crossover}(P_1, P_2)$ ;
7      $child \leftarrow \text{Mutation}(child)$ ;
8   else if  $B(p_c) = 1$  then
9      $P_1, P_2 \leftarrow \text{MateSelection}(Pop(t))$ ;
10     $child \leftarrow \text{Crossover}(P_1, P_2)$ ;
11   else if  $B(p_m) = 1$  then
12      $P_1 \leftarrow \text{MateSelection}(Pop(t))$ ;
13      $child \leftarrow \text{Mutation}(P_1)$ ;
14   if  $f(child) < (\max(f(P_1), f(P_2)) - c|f(P_1) - f(P_2)|)$  then
15      $Pop(t+1) \leftarrow Pop(t+1) \cup \{child\}$ ;
16      $i = i + 1$ ;
17   else
18      $POOL \leftarrow POOL \cup \{child\}$ ;
19    $p_{active} = \frac{i+|POOL|}{n_{pop}}$ 
20 while  $i < n_{pop}$  do
21   randomly select an offspring  $child_r$  from  $POOL$ ;
22    $Pop(t+1) \leftarrow Pop(t+1) \cup \{child_r\}$ ;

```

In Algorithm 2, a new offspring *child* is generated through a normal mate selection and crossover/mutation operators. The mate selection operator can be any canonical mate selection operator, like tournament selection or roulette wheel selection.

A comparison factor $c \in [0, 1]$ controls the threshold to determine whether an offspring *child* can survive or not, where the threshold is within a closed interval between the parents' fitness values $f(P_1)$ and $f(P_2)$ at line 7³.

An offspring *child* is added to offspring population $Pop(t+1)$ if it meets the criterion. Otherwise, this offspring *child* will be added to *POOL* at line 11, which collects the “discarded” offspring by the *offspring selection*. The discarded population pool *POOL* is initialized as an empty set in the first generation and is cumulatively updated in the later generations.

Parameter r defines the target *success ratio*, i.e. the ratio of the new population that should be filled with individuals that are better than their parents (according to the offspring selection criterion, line 4 in Algorithm 2).

Selection pressure limit p_{sel} represents a ratio of the maximum number of generated offspring to a population size n_{pop} . We call the parameter setting $c = 1$ and $r = 1$ *strict OS* because $\forall i = \{1, 2, \dots, n_{pop}\}, f(child) < \min(f(P_1), f(P_2))$. More variants of the *offspring selection* and

their impacts and influence on various problems were well explained and studied in Affenzeller et al. (2009, 2014).

2.3 Age layer population structure based GP

Age-layered population structure (ALPS) was proposed by Hornby (2006) as an approach to promote diversity via niching and periodic reinitialization. ALPS achieves this goal by introducing a new property, age, for every individual. A set of simple age update rules define how individuals can “grow old”:

- A new randomly initialized individual is assigned an age value of zero.
- Offspring produced by crossover or mutation inherit the age of the oldest parent.
- An individual's age is updated by an increment of one upon fitness evaluation.

The working principle in ALPS is then to restrict competition for survival only between individuals within the same *age layer*, in order to prevent “young” individuals from being driven to extinction by older individuals (that have had time to evolve better fitness). This form of niching promotes diversity by making the competition for survival “more fair”.

A new parameter *AgeGap* is introduced to define the age layers of the population. An age layer i is expressed as a half-closed integer interval:

³ For maximization problems, the criterion at line 14 can simply be inverted.

$$L_i = [\underline{\text{age}}^{(i)}, \overline{\text{age}}^{(i)}], \text{ where}$$

$$\underline{\text{age}}^{(i)} = \begin{cases} \overline{\text{age}}^{(i-1)} & \text{if } i > 0 \\ 0 & \text{otherwise} \end{cases}$$

$$\overline{\text{age}}^{(i)} = \text{AgeLimit}(i, \text{AgeGap})$$

The function *AgeLimit* is a user-defined function that defines the age intervals separating the layers and usually increases as the increment of layers by different strategies (e.g., linear, Fibonacci, polynomial or exponential). The layer hierarchy is defined from the bottom up, such that the youngest age layer (or “bottom” layer) will be at the bottom of the hierarchy while the oldest layer will be at the top. The entire population of ALPS is the union of the populations in all active layers.

Initially, there will be a single age layer. However, age update rules will quickly lead to a heterogeneous distribution of age values. Once an individual’s age exceeds the maximum age of a layer, the individual will be pushed to the next layer and a new individual will be inserted in its place (either transferred from the previous layer, if it exists, or randomly initialized). In addition, the bottom layer will be periodically reinitialized with new individuals, every *AgeGap* generations. This ensures that new genetic material periodically trickles into the process, making its way up the layer hierarchy and ensuring that diversity levels are maintained during the evolution.

As a result, ALPS is a unidirectional hierarchical structure that pushes individuals from a bottom layer to a top layer. This idea is different from the Hierarchical Fair Competition structure (HFC) Hu and Goodman (2002) which uses fitness information to segregate layers. It can be argued that age segregation in ALPS is more fair as it doesn’t consider fitness.

The Pseudocode of ALPS is shown in Algorithm 3. ALPS starts with the initialization of its parameters (line 1 - 3). The main loop of the ALPS framework (line 4 - 29)

cycles through each active layer and creates a new layer until a termination criterion is satisfied at line 4. Some common stop criteria include the number of function evaluations, the maximum number of active layers, and etc. In the main loop, a child’s age is updated (line 10). If this child’s age does not exceed the age limit in this specific layer, this child will survive in the current layer. Otherwise, this child will be removed from the current layer and be used to initialize (lines 13–15) or update the population (17–20) in the next layer. The next will be active when the number of individuals in the next layer equals the population size (lines 24–25). Once the main loop terminates, the optimal solution T^* is selected from the union of all layers.

Any EA can be integrated with ALPS, for example, to produce ALPS with GA (Hornby 2006; Fleck 2015), ALPS with GP (Patel and Clack 2007; Mateiu 2019) and ALPS with NSGA-II (Lichtberger 2019) for multi-objective optimization problems. The age-based segregation into layers in ALPS is a form of *niching*, a technique commonly used for diversity preservation in EAs. Other popular niching techniques in GP are:

- fitness-based, using explicit (Goldberg et al. 1987) and implicit (Smith et al. 1993) fitness sharing, clustering (Yin and Gerday 1993) or clearing (Pérowski 1996; Singh and Deb 2006)
- replacement-based, using deterministic (Mahfoud 1996), probabilistic (Mengshoel and Goldberg 1999) or local tournament-based (Mengshoel and Goldberg 2008) crowding or restricted tournament selection (Harik 1995)
- preservation-based, using species conservation (Li et al. 2002)
- hybrid, which implement adaptive versions or ensembles of fitness-, replacement- or preservation-based methods (Leung and Liang 2003; Yu and Suganthan 2010).

Algorithm 3: Pseudocode of ALPS

Input: parameters of any EA $para_{EA}$, objective function $f(\cdot)$, age gap for ALPS $AgeGap$;
Output: optimal solution T^*

```

1   $n_{alps} = 1$ ; ▷ Number of active layers
2   $Layer(0) \leftarrow \text{InitializePopulation}(para_{EA})$ ;
3   $idx = 0, t = 0$ ;
4  while not terminate do ▷ Main loop
5      for  $i = 0, i < n_{alps}, i++$  do ▷ Updating layers
6           $Layer(i)' = \emptyset$ ; ▷ Offspring population for the  $i$ -th layer
7          for  $j = 0; j < n_{pop}; j++$  do ▷ Updating one layer
8               $operator \leftarrow \text{SelectGeneticOperator}(para_{EA})$ ;
9               $P_1, P_2 \leftarrow \text{MateSelection}(Layer(i))$ ;
10              $child \leftarrow operator(P_1, P_2)$ ;
11              $\text{UpdateAge}(child)$ ;
12             if  $\text{Age}(child) > \text{AgeLimit}(i, \text{AgeGap})$  then
13                 if  $|Layer(i+1)| < n_{pop}$  then
14                      $idx = idx + 1$ ;
15                      $Layer(i+1) \leftarrow Layer(i+1) \cup \{child\}$ ;
16                 else ▷ The next layer is active
17                      $Layer(i+1) \leftarrow Layer(i+1) \cup \{child\}$ ;
18                      $T_d \leftarrow \arg \min_{T \in Layer(i+1) \wedge f(T) > f(child)} [\text{Age}(T)]$ ;
19                      $Layer(i+1) \leftarrow Layer(i+1) \setminus \{T_d\}$ ;
20                     if  $\text{Age}(T_d) < \text{Age}(child)$  then
21                          $Layer(i+1) \leftarrow Layer(i+1) \setminus \{T_d\}$ ;
22                          $Layer(i+1) \leftarrow Layer(i+1) \cup \{child\}$ ;
23                  $\text{Add a newly randomly generated child to } Layer(i)'$ ;
24             else
25                  $Layer(i)' \leftarrow Layer(i)' \cup \{child\}$ ;
26             if  $idx = n_{pop} - 1$  then
27                  $n_{alps} = n_{alps} + 1$  and  $idx = 0$ ; ▷ Activate the next layer
28              $\text{Select}(Layer(i) \cup Layer(i)')$ ;
29          $t = t + 1$ ;
30         if  $(t \bmod \text{AgeGap}(0) = 0)$  then
31              $Layer(0) \leftarrow \text{InitializePopulation}(para_{EA})$ ;
32  $T^* \leftarrow \arg \min f(T)$  where  $T \in \{Layer(0) \cup \dots \cup Layer(n_{alps})\}$ ;

```

The form of niching implemented by ALPS belongs to the category of preservation-based methods. Burks and Punch (2017) compared ALPS with other preservation-based methods (e.g., Age-Fitness Pareto optimization algorithm (Schmidt and Lipson 2011), subtree semantic geometric crossover (Nguyen et al. 2016), and genetic marker diversity algorithm for GP (Burks and Punch 2015)) and found that ALPS has the highest behavioral diversity in the entire population. However, a detailed analysis of the population diversity in each layer is not available in Burks and Punch (2017).

The algorithms presented in this section represent conceptually sound augmentations of canonical GP meant to improve its performance by promoting – and more efficiently exploiting – population diversity. However, the resulting changes and their respective impact on the algorithm's convergent behavior cannot be easily compared (for instance by attempting to replicate identical benchmarking conditions, using a fixed evaluation budget or fixing as many common parameters as possible). The effects produced by changes in the internal mechanisms of

selection or the topology of the population need to be considered within a methodological framework able to perform a detailed analysis both from a diversity and a genealogical perspective.

In the following, we introduce an analysis methodology capable of producing a detailed description of the evolutionary process, in terms of diversity (structural and semantic), inheritance, subtree frequencies and building blocks.

3 Analysis of evolutionary dynamics

3.1 Population diversity

In Sect. 1, we have argued the importance of understanding dynamical aspects of GP and the deep connection between algorithm convergence and population diversity. Many previous works used a single measure to describe diversity (e.g., structural/distance-based, behavioral, based on the number of unique individuals, genetic markers, etc.).

In order to have a more complete picture of the interplay between selection and recombination, in this work, we use two complementary measures (genotypic and phenotypic), corresponding to the two levels of interaction between genetic operators: selection acting on fitness at phenotype level, crossover and mutation acting on tree syntax at genotype level.

We employ computationally efficient methods for computing diversity. Our methods are normalized between $[0, 1]$ and have a natural correspondence to the concept of *similarity* between two trees. This in turn also allows for a more natural graphical representation of diversity loss in the empirical section, using a quantity that increases over time. We use the notation $Sim_g(\cdot)$ and $Sim_p(\cdot)$ to denote genotypic and phenotypic similarities, respectively. These similarities can be equivalently formulated as distances by simply subtracting the respective quantities from 1 (e.g., taking $1 - Sim_g(\cdot)$ and $1 - Sim_p(\cdot)$ as genotypic and phenotypic distances, respectively).

Since the measures are symmetric, the similarity matrix associated with the population is triangular and only $\frac{n_{pop}(n_{pop}-1)}{2}$ similarities must be computed. The average similarity $\overline{Sim}$ of is defined as the average of the similarity matrix. This definition applies to both genotypic similarity and phenotypic similarity.

3.1.1 Genotypic similarity

For genotypic diversity, we use a tree isomorphism algorithm which works in a bottom-up fashion similar to Valiente (2001). However, in our case, we resort to hashing (Burlacu et al. 2019), a more efficient algorithmic approach in which each tree node is assigned a unique hash value determined by its structure. Subtree isomorphism is then reduced to finding nodes with the same hash value.

We employ the *Jaccard index* to obtain a similarity value knowing the number of matching nodes between two trees.

$$Sim_g(T_i, T_j) = \frac{|M(T_i, T_j)|}{|T_i| + |T_j| - |M(T_i, T_j)|} \quad (3.1)$$

where $|M(T_i, T_j)|$ is the size of the intersection of T_i and T_j , M represents a mapping of isomorphic subtrees from T_i to T_j , and $|T_i|, |T_j|$ are the lengths of the two trees. Note that Jaccard similarity is a metric.

3.1.2 Phenotypic similarity

We compute phenotypic similarity between two tree models as the squared Pearson correlation coefficient between their responses on the same training data D . As the correlation is not defined when the standard deviation is

zero for either one of the responses, we introduce a couple of extra rules to make sure that the similarity measure is well defined:

- if both standard deviations are zero, then the similarity is one.
- if only one of the standard deviations is zero, then the similarity is zero.

This is formalized below. Let σ_i and σ_j be the standard deviations of the two tree models T_i and T_j :

$$Sim_p(T_i, T_j) = \begin{cases} 1 & \text{if } \sigma_i = \sigma_j = 0 \\ 0 & \text{if } \sigma_i = 0, \sigma_j \neq 0 \text{ or } \sigma_i \neq 0, \sigma_j = 0 \\ r_{T_i, T_j}^2 & \text{otherwise} \end{cases} \quad (3.2)$$

As we are using the squared correlation coefficient, the value of $Sim_p(T_i, T_j) \in [0, 1]$.

3.2 Inheritance patterns and building block propagation

The propagation of subtrees from one generation to the next represents an important aspect of GP evolutionary search behavior. In this context, we consider building blocks to be highly fit subtrees that are inherited by offspring individuals and increase their frequency in the population.

The occurrence of building blocks can be investigated using a frequentist approach where the action of genetic operators (crossover, mutation, selection) is in a first phase recorded in the form of a genealogy graph such that mating events and their outcomes are represented as vertices and arcs (Burlacu et al. 2015a).

Mining this body of information in a second phase can lead to the discovery of inheritance patterns, common ancestries, or frequently sampled subtrees. We describe in the following a practical approach for constructing a genealogy graph containing this information, as well as analysis methods on the graph for the discovery of frequent schemas and building blocks.

3.2.1 Subtree tracing

In the context of subtree tracing, we use the term *fragment* $fg(T_i)$ to refer to an inherited subtree in T_i individual, which is passed on to T_i from one of its parents. *Fragments* are saved in the genealogy graph G together with positional information identifying them in the parent and offspring trees. The positional coordinates are represented using indices into the node sequence given by a prefix traversal of the respective tree.

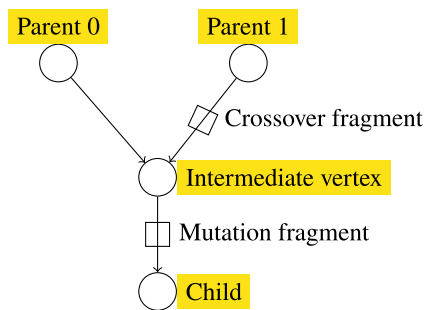


Fig. 2 Genealogy graph vertices representing a crossover operation followed by mutation

The storage scheme shown in Fig. 2 describes a situation where crossover and mutation are applied in succession. In this case, the offspring generated by crossover represents an intermediate individual that will not be present in the offspring population. In order to preserve continuity this intermediate individual is included as a vertex in the genealogy graph together with the respective fragments. Using the prefix indices stored in the genealogy graph, all fragments inherited via crossover can be identified. In the case of mutation, new genes can be introduced spontaneously. Thus, we consider the case when there is an overlap between the fragment and the traced subtree. In this case it is still useful to follow the origin of the subtree at the site of mutation in order to gain a deeper understanding of effects of the genetic operators.

The concept is illustrated in Fig. 3 where two parent individuals participate in crossover. The prefix indices of the swapped subtree within the non-root parent and the child individual are recorded in the genealogy graph.

A genealogy graph offers the ability to traverse an individual's ancestry following the inherited fragments until the initial population is reached. Consequently, subtree tracing enables the possibility to identify for every subtree in the population the exact sequence of genetic operations that has led to its creation.

The logic for tracing any subtree in the population relies on a set of simple arithmetic rules for processing prefix indices according to the relationship between the subtree being traced T and the fragment $f_g(T)$, respective to the containing tree individual T . Table 1 illustrates these rules and provides the foundation for a recursive procedure that navigates the genealogy graph and constructs a so-called *trace graph*, containing in its vertices only those ancestor individuals of the current tree which have contributed a genetic fragment to its genotype.

A trace graph can be seen as a compact representation of the inheritance information aggregated with respect to a given subtree. If we consider an individual's genealogy graph $G = (V, A)$, where V is the set of vertices (representing individuals) and A is the set of arcs (representing inheritance relationships), and an individual's trace graph $G_T = (V_T, A_T)$, the following are true:

- G is a digraph and G_T is a multidigraph. The reason for this is the existence of inbreeding in the GP population. Then, we have multiple intersecting evolutionary pathways tracing down to common ancestors, where multiple arcs between a pair of vertices represent trace paths tracing different fragments.
- $V_T \subset V$. For the purpose of compactness, the trace graph only includes ancestors from which elements of

Fig. 3 Schematic of a crossover representation. Prefix indices are shown under each node

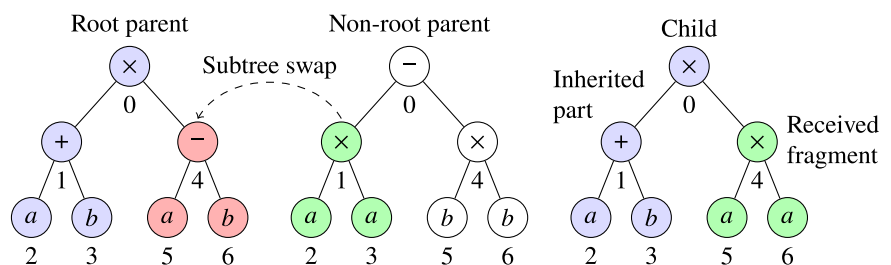


Table 1 Prefix indices arithmetic rules for subtree inclusion. Here, $f_g(T)$ is a tree fragment, T is the traced subtree, the $\mathcal{I}(\cdot)$ operator returns the prefix index and $\mathcal{L}(\cdot)$ returns the length of the fragment or subtree

1.	$\mathcal{I}(f_g(T)) = \mathcal{I}(T)$	Fragment and subtree coincide
2.	$\mathcal{I}(T) + \mathcal{L}(T) < \mathcal{I}(f_g(T))$ or $\mathcal{I}(f_g(T)) + \mathcal{L}(f_g(T)) < \mathcal{I}(T)$	Fragment and subtree are distinct
3.	$\mathcal{I}(f_g(T)) < \mathcal{I}(T) < \mathcal{I}(f_g(T)) + \mathcal{L}(f_g(T))$	Subtree contained by fragment
4.	$\mathcal{I}(T) < \mathcal{I}(f_g(T)) < \mathcal{I}(T) + \mathcal{L}(T)$	Subtree contains fragment

the genotype of the traced individual originated, without considering the propagation of said elements due to selection.

- A is a set of ordered vertex pairs from V , while A_T is a multiset of ordered vertex pairs from V_T . This is explained by the fact that single ancestor individuals generate multiple descendants who become subsequently intermingled, leading to intercrossing inheritance patterns.

3.2.2 Frequency-based identification of building blocks

For reasons enumerated above, the decomposition of a traced subtree⁴ results in a record of genotypic operations (crossover, mutation) and associated set of genetic fragments transferred from ancestor to descendant individuals. We can then quantify the propagation of subtrees in the GP population by applying the tracing algorithm to the entire population and counting the number of times each subtree was sampled by a genetic operator.

Algorithmically, the counting is performed by the tracing procedure by incrementing a counter for all the nodes of each visited subtree. Since the incrementing scheme leads to child node counts that are always greater or equal than parent node counts, we define a node's *sample count* as the difference between its own count and the count of its parent, as illustrated in Fig. 4.

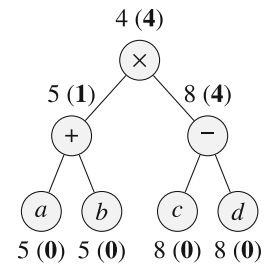
Together with this node numbering scheme, the tracing algorithm can directly quantify the sampling frequencies of all the subtrees in the GP population via the calculated sample counts. This allows the discovery of building blocks as the most frequently sampled subtrees in the population.

3.2.3 Contribution ratio

Previous work has already shown that the percentage of ancestor individuals having descendants in the last generation is low and that only a small fraction of ancestors contribute genetic material to the solutions in the last generation (Donatucci et al. 2014; McPhee et al. 2015, 2016). We quantify the fraction of contributing ancestors as the ratio between the size of the trace graph (in which vertices represent ancestors that have contributed a part of their genotype to their descendants in the final population) and the size of the genealogy graph. Additionally, the ratio of arcs to vertices in the trace graph indicates the number of times subtrees are traced to the same ancestor individual and can help identify the so-called progenitors in the genealogy. This represents a

⁴ We can perform tracing of an individual's entire genotype when the subtree to be traced is rooted in the tree individual's root node.

Fig. 4 Node counts (above each node) and *sample counts* (bold font, inside parentheses)



systematic approach of characterizing algorithm convergence, as low contribution ratios mean that most recombination events shuffled subtrees around in the population without making actual progress in terms of achieving adaptive change.

4 Empirical experiments

4.1 Test problems

We illustrate the dynamic analysis methods described in Sect. 3 on five symbolic regression benchmarks *Poly-10* (Poli and McPhee 2003), *GP-Challenge* (Widera et al. 2010), *Friedman-I* (Friedman 1991), *Friedman-II* (Friedman 1991), and *Aircraft Maximum Lift Coefficient* problem (Chen et al. 2018).

The test problems have been selected based on the following considerations. Firstly, they are well-established in the field of GP, additionally they are included (or voted to be included) into a standard benchmark set for symbolic regression (White et al. 2013) (GP-Challenge, Friedman spline regression problems). Secondly, problems where the structure of the target function is known (Poly-10, Friedman-I, Friedman-II, Aircraft) allow us to validate the methodology, by checking if subtree tracing finds building blocks that correspond to terms of the target function. The Poly-10 problem in particular makes it easy to compare the identified subtrees with the linear terms of the formula. The problems were also chosen to be the right level of difficulty for the chosen algorithmic settings.

4.1.1 Aircraft maximum lift coefficient

In aircraft design, the maximum lift coefficient of a whole aircraft can be approximately modeled as:

$$f(\mathbf{x}) = x_1 - 0.25x_4x_5x_6 \left[4 + 0.1 \frac{x_2}{x_3} - \left(\frac{x_2}{x_3} \right)^2 \right] \\ + x_{13} \frac{x_{14}}{x_{15}} x_{18} x_7 - x_{13} \frac{x_{14}}{x_{15}} x_8 + x_{13} \frac{x_{14}}{x_{15}} x_9 \\ + x_{16} \frac{x_{17}}{x_{15}} x_{18} x_{10} - x_{16} \frac{x_{17}}{x_{15}} x_{11} + x_{16} \frac{x_{17}}{x_{15}} x_{12}$$

where $x_1 \in [0.4, 0.8]$, $x_{4,13,16} \in [2, 5]$, $x_2 \in [3, 4]$, $x_3 \in [20, 30]$, $x_{8,11,14,17} \in [1, 1.5]$, $x_{15} \in [5, 7]$, $x_{18} \in [10, 20]$, $x_{9,12} \in [1, 2]$, $x_{7,10} \in [0.5, 1.5]$, $x_5 \in [0, 20]$, $x_6 = 1$. The dataset consist of 200 samples, where the first 100 samples are used for training and the rest are use for the test.

4.1.2 Friedman-I and Friedman-II

Friedman-I contains 10 continuous predictor variables, where five variables are independent and the rest five variables are redundant. Friedman-I is defined as:

$$f(\mathbf{x}) = 0.1e^{4x_1} + \frac{4}{1 + e^{-20x_2+10}} + 3x_3 + 2x_4 + x_5$$

Friedman-II contains 10 continuous predictor variables including three non-linear and interacting variables, two linear variables, and five irrelevant variables. Friedman-II is defined as:

$$f(\mathbf{x}) = 10 \sin(\pi x_1 x_2) + 20(x_3 - 0.5)^2 + 10x_4 + 5x_5$$

Both Friedman-I and Friedman-II datasets consist of 10000 samples, where the first 5000 samples are used for training, and the rest samples are for the test.

4.1.3 GP-challenge

The GP-Challenge problem consists of data generated artificially from protein decoy energy terms, where the goal is to predict the protein energy function used for protein structure prediction (Widera et al. 2010). The data consists of 82965 data points, where the first 5000 data points are reserved for training and the rest of the data points are used as test data. The input feature set consists of the variables:

- $t_1, \dots, t_3$: three short-range potentials between C_α atoms E_{13} , E_{14} and E_{15} .
- t_4 : local stiffness potential E_{stiff}
- t_5 : hydrogen bonds potential E_{HB}
- t_6 : long-range pairwise potential between side chain centres of mass E_{pair}
- t_7 : electrostatic interactions potential E_{electro}
- t_8 : environment profile potential E_{env}
- RMSD: root mean square deviation between 3D coordinates of C_α atoms
- rank: rank of each decoy based on increasing order of RMSD

4.1.4 Poly-10

This problem is a synthetic benchmark problem where the target is defined as a sum of non-linear terms:

$$f(\mathbf{x}) = x_1 x_2 + x_3 x_4 + x_5 x_6 + x_1 x_7 x_9 + x_3 x_6 x_{10}$$

The data consists of 500 data points of which the first half are used for training and the rest are used as test data.

4.2 Parameter settings

The parameters for an individual are defined in Table 2. The parameter settings for GP, OSGP, and ALPS-GP are shown in Table 3 where p_m and p_c stand for mutation and crossover probabilities, respectively. The selection operator was parameterized for each algorithm according to suggested settings: tournament selection with group size 5 for GP, gender-specific selection (Affenzeller et al. 2005) for OSGP, and generalized rank selection (Tate and Smith 1995) for ALPS. The termination criterion for GP and ALPS-GP is 500 generations while for OSGP the termination criterion is a maximum selection pressure p_{sel} of 100. Each experiment consisted of 33 repetitions of the same configuration in order to account stochastic effects.

In this paper, the algorithms have not been limited in runtime or evaluation budget since each algorithm converges differently and our goal is to analyze the different resulting evolutionary dynamics. These limitations make sense in benchmarking scenarios where the goal is to obtain the best model within certain computational constraints, but in our case imposing the same limits would have hindered our ability to investigate the different behaviors.

4.3 Empirical results

We describe algorithm performance in terms of the *normalized mean squared error* (NMSE) of the best solution on the training and test datasets. Metrics based on the sum of squares (e.g. MSE, NMSE, RMSE) are typically used to assess model quality as they are more intuitive to understand (always non-negative, values close to zero are better). Note that the metric used here to describe results is not related to the fitness measure (Pearson's R^2) used by the algorithms. Within GP, it is recommended to use either correlation or mean squared error with linear scaling for fitness (Keijzer 2003).

$$\text{NMSE}(\hat{y}, y) = \frac{\text{MSE}(\hat{y}, y)}{\text{Var}(y)} \quad (4.1)$$

where $\hat{y}$ is the model prediction, y is the actual regression target, and $\text{Var}(y)$ is the variance of the target. The errors shown in Table 4 are expressed as median NMSE $\pm$ IQR over the 33 repetitions, where IQR is the interquartile range.

Statistical analysis based on a Kruskal test ($p < 0.05$) is shown in Table 5. On the complex problems with more

Table 2 Parameter settings for an individual T

$\mathcal{F}$	$\mathcal{TN}$	d	l	Initialization	f(.)
$+, -, \times, \div, \exp, \log, \sin, \cos, \sqrt{\cdot}, (^2)$	$\{c, x\}$	[1, 10]	[1, 50]	PTC2 (Luke 2000)	R^2

Table 3 Parameter settings for algorithms

Algorithm	n_{pop}	p_c	p_m	Selection	Termination
GP	1000	1.0	0.25	Tournament	$t > 500$
OSGP	500	1.0	0.25	GenderSpecific	$p_{\text{active}} > 100$
ALPS	300	1.0	0.25	GeneralizedRank	$t > 500$

Table 4 Modeling results expressed as median normalized mean squared error $\pm$ interquartile range

Problem	Algorithm	NMSE train	NMSE test
Aircraft	ALPS	0.0063 $\pm$ 0.0029	0.0056 $\pm$ 0.0040
	GP	0.0081 $\pm$ 0.0042	0.0079 $\pm$ 0.0045
	OSGP	0.0057 $\pm$ 0.0043	0.0065 $\pm$ 0.0079
Friedman-I	ALPS	0.1444 $\pm$ 0.0037	0.1532 $\pm$ 0.0041
	GP	0.1443 $\pm$ 0.0082	0.1537 $\pm$ 0.0071
	OSGP	0.1414 $\pm$ 0.0030	0.1513 $\pm$ 0.0030
Friedman-II	ALPS	0.0433 $\pm$ 0.0060	0.0431 $\pm$ 0.0071
	GP	0.0473 $\pm$ 0.0155	0.0479 $\pm$ 0.0152
	OSGP	0.0428 $\pm$ 0.0056	0.0426 $\pm$ 0.0058
GP-Challenge	ALPS	0.0618 $\pm$ 0.0040	0.0673 $\pm$ 0.0069
	GP	0.0651 $\pm$ 0.0115	0.0709 $\pm$ 0.0107
	OSGP	0.0576 $\pm$ 0.0094	0.0641 $\pm$ 0.0102
Poly-10	ALPS	0.1580 $\pm$ 0.0691	0.2083 $\pm$ 0.0960
	GP	0.1764 $\pm$ 0.1906	0.2374 $\pm$ 0.2665
	OSGP	0.1542 $\pm$ 0.0830	0.1966 $\pm$ 0.1056

than 10 relevant variables (e.g., the Aircraft and the Poly-10 problems), there exist small but significant differences between ALPS and GP. On Friedman-I and GP-Challenge problems, significant differences exist between ALPS and OSGP.

4.4 Dynamic analysis

In this section, we demonstrate how similarly-performing algorithms in terms of solution quality achieve their results through very different internal dynamics of the search process. Our findings can help users determine whether specific evolutionary models are more suited for solving certain kinds of problems. For example, we assume that:

Table 5 Statistical significance p values according to the Kruskal test

Algorithm	Problem	ALPS		GP	
		Train	Test	Train	Test
GP	Aircraft	0.2019	0.0252		
GP	Friedman-1	0.4081	0.6125		
GP	Friedman-2	0.1974	0.2065		
GP	GP-Challenge	0.1491	0.4081		
GP	Poly-10	0.0003	0.0016		
OSGP	Aircraft	0.2785	0.4846	0.0536	0.1974
OSGP	Friedman-1	0.0000	0.0005	0.0261	0.0317
OSGP	Friedman-2	0.4378	0.5340	0.0421	0.0475
OSGP	GP-Challenge	0.0005	0.0004	0.0006	0.0016
OSGP	Poly-10	0.8625	0.6957	0.0035	0.0168

- ALPS would be more suitable for dynamic environments characterized by dynamic changes in the operating regime of the analyzed system. In such cases, the constant influx of new genetic material due to the periodic reinitialization of layer 0 would enable the algorithm to react to these changes meaningfully.
- OSGP would be more suitable for difficult problems with a high risk of premature convergence if certain alleles run extinct before they can be combined into meaningful building blocks. This is because the more focused exploitation of existing diversity by offspring selection minimizes the risk of losing important alleles.

As extensions to canonical version, both ALPS and OSGP are expected to perform equal or better than standard GP. This kind of analysis helps better understand the effects of certain parameterizations on run dynamics and can lead to algorithmic developments where aspects like selection pressure, comparison factor (OSGP), age gap (ALPS), etc. could be dynamically adjusted in order to optimally exploit the properties of the problem and the fitness landscape.

First, we illustrate the differences in the evolution of population diversity and solution quality. Then, we illustrate the tracing methodology and enumerate the subtrees ranked according to their sample counts calculated by the tracing procedure. Note that, in order to improve the flow of the paper, the referenced figures are found in the “Appendix”.

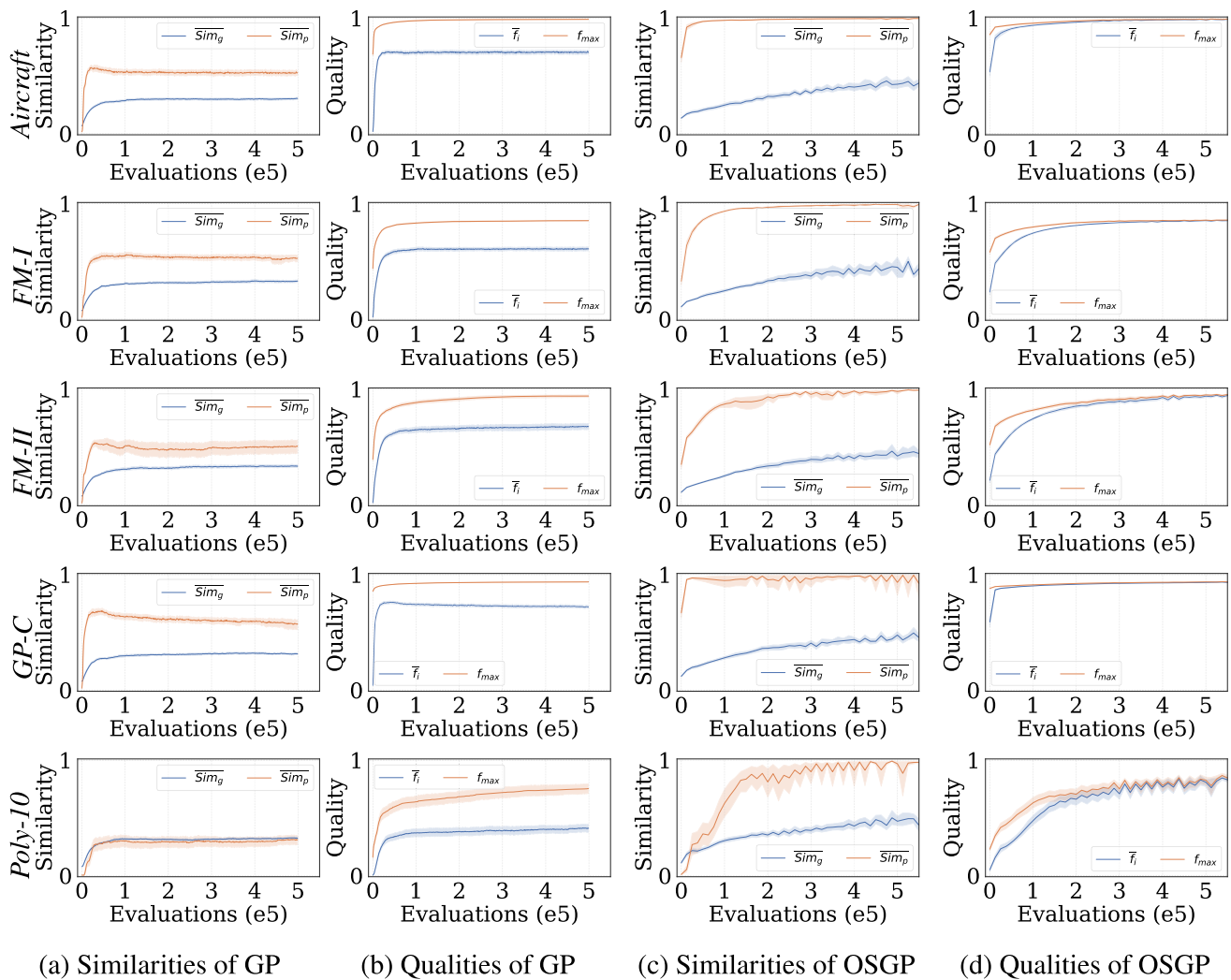


Fig. 5 Similarities and qualities of the GP and the OSGP on the five test problems

4.4.1 Quality and similarity

In this section, we analyze the dynamic analysis w.r.t. convergent similarities and qualities. Besides, the evolution of the joint distributions over genotype and phenotype similarities are also analyzed for the GP and OSGP.⁵

4.4.2 GP and OSGP

Figure 5 represents the dynamic analysis of similarity and model quality on the *Aircraft*, *Friedman-I*, *Friedman-II*, *GP-Challenge* and *Poly-10* problems. The plots illustrate the evolution of average *phenotype* and *genotype*

similarities alongside with the best and average fitness (1 - NMSE). The first two columns correspond to GP while the last two columns correspond to OSGP. The values are aggregated over the 33 algorithm repetitions. Since OSGP runs are adaptively terminated according to a maximum selection pressure criterion ($p_{sel} = 100$), the number of generations can vary in each run. For this reason, similarity and quality values for OSGP are aggregated over the 33 repetitions up to the median generation number, leading to some instability as observed in the right hand side of the charts.

An important observation here is that although phenotype and genotype similarity values are represented on the same graph and their respective metrics are in the same range, these are not directly comparable since they measure diversity on different levels. On the phenotype level, since solution candidates are selected based on the “similarity” of their predictions to the target variable (according to the

⁵ The evolution of the similarities for ALPS are theoretically doable but are not presented in this paper. Because the ALPS has multiple layers and showing everything in the same plots together with the similarities from GP and OSGP would make things too unclear.

fitness measure), an increase in phenotype similarity is expected as a side effect of the optimization process.

The differences between GP and OSGP are explained by the additional offspring selection step in OSGP rejecting the outcome of recombination when the child is worse than its parents. In this way, deleterious changes to the parent genotypes are not accepted. This single rule leads to significant changes in the algorithm behavior.

In the case of GP, we note a pronounced loss of diversity in the first few generations, after which the value stabilizes as a result of structural homogeneity in the population balanced by a constant influx of new genetic material due to mutation. We note that after the initial diversity loss accompanied by a dramatic increase in fitness, the quality curves remain mostly flat. This supports the hypothesis that genetic improvement can only occur when enough diversity is present in the population. When this diversity is used up, the search begins to stagnate.

In the case of OSGP, the more focused selection strategy leads to a more gradual genotype similarity increase between offspring and their parents, noticeable on problems from the first and the third columns in Fig. 5, and a smoother transition between the exploratory and the exploitative phases of the search. This suggests that OSGP is better able to utilize the diversity in the population and as a consequence, does not rely on mutation that much. Strict offspring selection also causes phenotype similarity to increase towards 1, as only offspring with an improved fitness (compared to their parents) are accepted.

The almost identical shape of the genotype similarity curves for each problem suggests that structural diversity depends rather more strongly on selection pressure than on the characteristics of the fitness landscape. On the other hand, phenotype similarity is clearly influenced by the problem characteristics, as observed for both GP and OSGP. The evolution of average similarity curves already highlights some important differences between the tested algorithms. However, averages tend to obscure the real distribution of similarity values in the population.

The joint distributions of genotype and phenotype similarity values on the five tested problems are shown for GP and OSGP in Figs. 6, 7, 8, 9 and 10. The snapshots shown in these figures represent joint probability plots aggregated from similarity points from all 33 repetitions. For canonical GP, in each figure, from left to right, we showed 4 snapshots of the joint distributions of the populations with 2%, 36%, 70% and 100% function evaluations. For OSGP, since run lengths are not uniform, the number of evaluations is extrapolated from the selection pressure per generation, as we consider selection pressure a better indicator

of convergence status. The hexbin color indicates point density and the x and y coordinates are calculated as an individual's average similarity with the rest of the population, by taking the row-wise averages of the genotype and phenotype similarity matrices, respectively.

The shape of the univariate distributions (or the marginal distributions of the multi-variate distribution) of the two similarities resembles a normal curve as a result of the central limit theorem. The observed distributions correlate with our interpretation of how GP and OSGP make use and exploit genetic diversity in the population. Requiring adaptive change via offspring selection causes an abrupt increase in phenotype similarity. In other words, selecting strictly for *adaptive* change lowers the diversity of the overall behaviors in OSGP. This brings in turn more variance in tree structures, as indicated by the spread-out shape of the genotype similarity marginal distributions. For GP, the distribution of phenotype similarities is left-skewed, with a long tail in the area of high behavioral diversity, meaning that although most individuals exhibit similar behavior, some individuals are diverse, likely due to lower fitness. The distribution of genotype similarities is normal-shaped and similar across problems.

From these charts, we can conclude that an evolutionary model focused on adaptive change (OSGP) leads to a more intense focus on building blocks, as the building block hypothesis implicitly relies on parents' ability to produce children of comparable or even better fitness. This focus in turn leads to decreased phenotype diversity as behaviors are less likely to deviate from the parents and more likely to be similar between descendants.

4.4.3 ALPS

The dynamic analysis of the ALPS on the problems are shown from Figs. 11, 12, 13, 14 and 15, respectively. A periodic pattern for both the genotype and the phenotype similarities occurs in the layer L_0 because the bottom layer is re-initialized every 20 generations to ensure a constant influx of genetic material in the population. The population re-initialization at the bottom layer affects other layers by introducing a periodic pattern. However, this periodic effect diminishes gradually in the upper layers, because a larger age gap exists in the higher layers which allows the layer to converge to a "stable" population. Therefore, the top layer (L_5 in this case) is the most stable layer in terms of the genotype and the phenotype changes. Furthermore, compared with similarities of the GP and the OSGP in Fig. 5, the top layer L_5 still possesses the most diverse population in the steady stage. The periodic pattern of the

similarities also occurs in quality analysis in terms of the best quality and the average quality in a specific layer population. The best quality of every layer diminishes because of unidirectional movement of an individual from a lower layer to a upper layer. The decrease of the best quality in the top layer L_5 does not occur as there is no individual movement in the top layer.

4.4.4 Subtree frequencies and building block identification

We employ the methodology described in Sect. 3.2 to calculate subtree frequencies for the GP, OSGP and ALPS algorithms. In the case of ALPS, a genealogy graph is generated for each layer. Due to the unidirectional movement of individuals as they age, the layer genealogy graphs overlap to a significant degree.

For each algorithm we select the best solution out of the 33 repetitions based on the normalized mean squared error. Then the tracing methodology is applied to the population, according to the following steps:

1. The genealogy graph is constructed online during the evolution.

2. A trace graph is calculated for the entire final population in a post-analysis step. During tracing, subtree sample counts are updated as described in Sect. 3.2.2.
3. A post-processing step hashes each subtree ignoring coefficients for terminal nodes in order to focus on structure. This decision was made since the leaf coefficients are real-valued and including them in the hashing would have generated too many variations. The sample counts of isomorphic subtrees are then added together in order to obtain the final ranking.

The resulting genealogy graphs are illustrated in Table 6. The number of vertices $|V|$ is determined by the population size, number of generations and mutation probability. For GP and OSGP, we have:

$$|V| = n_{pop} + (n_{pop} \cdot (p_c + p_m)) \cdot n_{gen} \quad (4.2)$$

In the case of ALPS, due to the layered population structure, the number of vertices $|V|$ given by 4.2 is increased by the term $n_{pop} \cdot AgeGap$. Additionally, vertices are not removed from the graph when corresponding individuals are moved out of the layer, because the information is

Table 6 Genealogy graph sizes for the tested configurations. The first line represents the number of vertices and the second line represents the number of arcs

	Aircraft	Friedman-1	Friedman-2	GP-Challenge	Poly-10
GP	$ V = 625,591$	$ V = 625,763$	$ V = 625,903$	$ V = 626,442$	$ V = 625,591$
	$ A = 1,124,451$	$ A = 1,124,263$	$ A = 1,124,403$	$ A = 1,124,942$	$ A = 1,124,451$
OSGP	$ V = 818,841$	$ V = 2,186,478$	$ V = 1,478,884$	$ V = 1,651,145$	$ V = 1,696,312$
	$ A = 1,473,241$	$ A = 3,934,678$	$ A = 2,661,184$	$ A = 2,971,345$	$ A = 3,052,712$
ALPS	$ V = 311,541$	$ V = 304,125$	$ V = 288,909$	$ V = 299,978$	$ V = 303,128$
	$ A = 340,339$	$ A = 339,922$	$ A = 340,286$	$ A = 340,463$	$ A = 340,103$

Table 7 Trace graph sizes for the tested configurations. The first line represents the number of vertices and the second line represents the number of arcs

	Aircraft	Friedman-I	Friedman-II	GP-Challenge	Poly-10
GP	$ V = 12,591$	$ V = 17,574$	$ V = 17,139$	$ V = 14,059$	$ V = 10,477$
	$ A = 35,720$	$ A = 52,301$	$ A = 49,808$	$ A = 42,236$	$ A = 31,808$
OSGP	$ V = 65,153$	$ V = 65,088$	$ V = 64,919$	$ V = 65,621$	$ V = 65,233$
	$ A = 1,27,123$	$ A = 1,25,699$	$ A = 1,29,091$	$ A = 1,27,552$	$ A = 1,30,035$
ALPS	$ V = 6881$	$ V = 6847$	$ V = 7394$	$ V = 6946$	$ V = 6881$
	$ A = 10,567$	$ A = 10,984$	$ A = 12,704$	$ A = 11,947$	$ A = 11,146$

Table 8 Solution contribution ratios for the tested configurations

	Aircraft (%)	Friedman-1 (%)	Friedman-2 (%)	GP-Challenge (%)	Poly-10 (%)
GP	2.0	2.8	2.7	2.2	1.7
OSGP	8.0	3.0	4.4	4.0	3.9
ALPS	2.2	2.3	2.6	2.3	2.3

necessary for computing an accurate history. Therefore, the upper layers grow larger in size, since they will additionally include the vertices corresponding to the individuals that are being moved from the previous layer after aging. For this reason, the graph size reported for ALPS in Table 6 are those corresponding to the final layer (Layer 5).

The differences between the theoretical limits and the values reported in Table 6 are due to stochastic effects. Table 8 shows the contribution ratios, defined in Sect. 3.2.3, of each algorithm on the five test problems. The value is calculated as the ratio between a trace graph's size and a genealogy graph's size. In the case of ALPS, the final population solutions are traced in each layer, and the results are aggregated together to obtain a total trace graph size which is then divided by the size of the genealogy graph in the last layer (since this graph already includes vertices for all the individuals migrated from the previous layers). The aggregated contribution ratio value is then comparable to the contribution ratios of the GP and OSGP algorithms.

According to Table 8, OSGP features the largest amount of contributing ancestors, followed by GP and then ALPS. This is consistent with the assumption that strict offspring selection determines a smoother and more gradual convergence, due to the more granular and effective utilization of the existing diversity. This can be also observed in the OSGP quality charts from Fig. 5 where the increase is more gradual compared to canonical GP.

Standard GP features a lower contribution ratio since the standard evolutionary model makes it easier for highly-fit "super individuals" to quickly dominate the population in subsequent generations, becoming the ancestors of all the descendants recorded in the genealogy graph. A lower contribution ratio corresponds to a smaller region of the genotype space sampled by the genetic operators during the algorithm's attempt to produce adaptive change. This region is continuously reduced by loss of diversity (every generation), up to the point where fitness is locally maximized and the available pool of genetic material is limited to a reduced common set of inherited genes, originating from a small number of "super ancestors".

ALPS features contribution ratios similar with GP, although in this case we have to take into account the fact that many new individuals are introduced in the genealogy graph by the reseeding of the bottom layer. This suggests that ALPS layers and the standard GP population evolve in a similar way, with the added benefit of a constant influx of diversity for ALPS that allows it to avoid premature convergence. Therefore, instead of evolving a single large

population, it seems more beneficial to evolve a number of smaller populations structured into age groups.

Overall, our results confirm previous observations (Kuber et al. 2014; Donatucci et al. 2014; McPhee et al. 2016, 2017) that only a small fraction of ancestor individuals actually contribute genetic material to the best solution. GP algorithms can potentially be made more robust by enforcing a more granular and less wasteful (in terms of diversity) exploitation of the alleles present in the randomly initialized starting population. Niching techniques, based for instance on age or inheritance information could be used to ensure that diversity is used more efficiently during the search.

4.4.5 Building blocks

Empirical measurements show that the most frequently sampled subtrees originate from ancestor individuals identified by the tracing methodology. Compared with the mathematical expression of the regression target, the identified subtrees include all the relevant input variables. However, since selection in GP does not consider expression complexity, the algorithms have a tendency towards more complex expressions, semantically equivalent to the corresponding formula terms.

Tables 10, 11, 12, 13, 14 and 15 in the "Appendix" show the subtrees identified by each algorithm on the five test problems in Sect. 4.1. The top 10 identified subtrees and their corresponding sample counts are represented in these figures. Table 9 is a summary of Tables 10, 11, 12, 13, 14 and 15 and shows the identified subtrees that correspond to the test problem formulas, except for the *GP-Challenge problems*, as there is no explicit formula for this problem. A subtree T is in the bold format if T exactly matches one term or a sub-term of one term in the problem formula.

The difference in the actual frequencies reported for each algorithm are determined, on the one hand, by differences in each algorithm's parameters (e.g., population size, maximum number of generations, selection scheme) and, on the other hand, by differences in their inherent search behavior. For this reason, subtree sample counts cannot be directly compared between algorithms, and instead we compare the relative ranking. We do observe, however, an inverse correlation between an identified subtree's size and its sampling frequency, as smaller subtrees with a simpler structure tend to be the most sampled.

For clarity as well as space reasons, we focus our discussion on the *Poly-10* problem, since its target formula is more explicit and relatively simpler than the other test

problems (only involving addition and multiplication operations). The main difficulty in this synthetic benchmark problem consists in finding the appropriate three-variable products: $x_1x_7x_9$ and $x_3x_6x_{10}$. Since three of the variables (x_7 , x_9 , x_{10}) do not occur at all in the other three terms of the formula (x_1x_2 , x_3x_4 , x_5x_6), there is a non-trivial probability for them to go extinct from the population before the algorithm can assemble them into meaningful building blocks, leading to premature convergence.

Both GP and OSGP are able to discover all three two-variable terms of the formula but find only of the two three-variable terms. The term $x_3x_6x_{10}$ is not found therefore the two algorithms do not completely solve the problem. While GP samples shorter and more simple terms, OSGP tends to sample larger, more complex terms that can simplify to sums of two variable products. The larger and more complex building blocks assembled by OSGP are determined by offspring selection, which ensures that already existing good building blocks in the structure of the root parent are not disrupted by crossover or mutation. This leads us to conclude that GP exhibits more granular inherited patterns (smaller subtrees) while in OSGP these patterns become larger due to the more constructive approach leading to the reuse of fit alleles from parent individuals.

In the case of ALPS, the search becomes more focused on the terms of the formula as the population ages and individuals migrate to the upper layers. In the bottom layer, since the population is only partially converged due to the periodic reseeding, the most sampled subtrees are simple structures containing relevant variables not necessarily assembled in an optimal relationship. However, moving to the upper layers we notice that the identified subtrees gradually begin to approximate more exactly the terms of the target expression, growing in complexity until the last layer. In this case, the layered evolution is able to correctly assemble the building blocks of the problem, finding the term $x_3x_6x_{10}$ that the other two algorithms failed to find. In terms of the complexity of the sampled subtrees, ALPS behaves more like standard GP in the lower layers and more like OSGP in the upper layers.

In summary, the contribution ratio calculated on the basis of the genotypes in the population and their hereditary relationships gives a good indication of the amount of genetic diversity lost through the action of selection during the evolutionary search. At the same time, it acts as a measure of search efficiency or *yield* as the algorithm evolves towards the best solution, since it directly quantifies how many of the total genotype operations that acted on ancestors of the individuals in the final population had a direct contribution to their structure.

A high contribution ratio implies frequent adaptive changes across an individual's ancestry, corresponding to incremental improvement. Since the changes are adaptive (they improve fitness), they are maintained by selection and present in the trace graph. On the other hand, a low contribution ratio corresponds to neutral or deleterious changes which are not fixated by selection and may get rejected or simply drift away.

Empirical observation shows that once diversity goes below a certain threshold, most of these operations fail to produce any meaningful new improvement, since the exchanged genetic material (e.g. by crossover) is already predominant in the population. From this perspective, we can rank the algorithms in terms of search efficiency – purely from the perspective of genetic convergence – as ALPS being the most efficient, followed by OSGP and lastly by GP. In practice, improving *yield* often comes at the cost of increased energy expenditure, analogous in this case with the spent evaluation budget.

The observed structures and frequencies of the identified building blocks represent additional evidence in support of the idea that a more efficient search is more easily able to assemble increasingly complex building blocks, as is the case for OSGP and ALPS.

5 Conclusions and future work

This paper analyzed population diversity and genealogy aspects for canonical GP, offspring selection GP (OSGP) and age-layered population GP (ALPS) in terms of population diversities, building block occurrences, and subtree inheritance patterns from the initialized population to the final population. The experimental benchmark set consisted of five different test problems, including three artificial benchmarks and two real-world problems.

The experimental results reveal the different evolutionary search characteristics of the three GPs. More specifically, canonical GP extracts diversity from the population comparatively fast and maintains the search mainly via mutation. Therefore, canonical GP flavors shorter subtrees. On the other hand, OSGP ultimately leads to a less diverse population because of the strict criterion in the offspring selection, but the rate of diversity loss is reduced as the convergence is smoother. The OSGP relies comparatively less on the mutation to maintain and assemble relevant building block information. Therefore, OSGP tends to produce longer subtrees, and the contribution ratios of the OSGP are the highest among all the three GPs. ALPS exhibits a unique character due to the continuous transfer

of every new genetic diversity across the younger age layers. Due to the periodic population initialization, a periodic pattern of diversity evolution occurs in the bottom layer. This periodic pattern is less pronounced as the layer number is increased. Regarding identified subtrees, its behavior is similar to that of the canonical GP in the lower layers (e.g., layer 0 and layer 1). In higher layers, ALPS trends to longer subtrees because of the cumulative effect of fitness comparison when an individual moves to the next layer in the previous layers.

The proposed similarity metrics can be efficiently implemented with minimal overhead such that the corresponding quantities can be computed during the evolutionary run. The combination of genotype and phenotype measures helps paint a more complete picture of diversity loss and reveal the different characteristics of the evolutionary search between different GP flavors. The tracing algorithm based on information provided by a genealogy graph represents a useful instrument for identifying relevant solution building blocks and characterizing the efficiency of the evolutionary search. A straightforward way to build the genealogy graph during evolution is by instrumentation of the recombination operators such that each

new offspring is inserted into the graph together with relevant inheritance information. The analysis proposed in this paper reveals different inheritance patterns between the tested algorithmic variants as well as between problems solved by the same algorithm, supporting the idea that search behavior is highly dependent not only on algorithm parameterization but also on problem characteristics.

For future works, it will be interesting to investigate strategies for dynamically adjusting hyperparameters based on observed dynamical aspects, for example the distribution of similarity values and building block frequencies during the search process. In addition, it is also worth to try to improve the effectiveness of the GPs by reducing the space of functional operators $\mathcal{F}$. Space $\mathcal{F}$ can be reduced by identifying/counting the useful functional operators in the most sampled subtrees and removing the “irrelevant” operators that never occur in the identified subtrees.

Appendix

See Tables 9, 10, 11, 12, 13, 14, 15, 16 and Figs. 6, 7, 8, 9, 10, 11, 12, 13, 14, 15.

Table 9 Identified subtrees that correspond to the problem formulas

Alg.				
<i>Aircraft</i>				
GP	$[x_5 * x_4]_-$	$[x_{13} * x_7 * x_{18}]_-$	—	—
OSGP	$[x_5 * x_4]_-$	$[x_{17} * x_{18}]_-$	$[1/x_{15}]_-$	$[x_{13} * x_{17}]_-$
ALPS	$[x_5 * x_4]_-$	$[x_5]_-$	$[x_{10}]_-$	—
<i>Friedman-I</i>				
GP	—	—	—	—
OSGP	$[1/\exp(x_2)]_{\approx}$	$[\exp(x_2)]_{\approx}$	—	—
ALPS	$[x_{10}]_-$	$[\exp(2 * x_1)]_{\approx}$	—	—
<i>Friedman-II</i>				
GP	$[x_1 * x_2]_-$	$[x_3^2]_-$	$[\sin(x_1 * x_2)]_{\approx}$	—
OSGP	$[x_1 * x_2]_-$	—	—	—
	$[x_1 * x_2]_-$	$[x_2]_-$	$[x_5]_-$	$[x_3^2]_-$
ALPS	$[\sin(x_1 * x_2)]_-$			
<i>Poly-10</i>				
GP	$[x_3 * x_4]_=$	$[x_5 * x_6]_=$	$[x_1 * x_2]_=$	$[x_1 * x_7 * x_9]_=$
OSGP	$[x_3 * x_4]_=$	$[x_5 * x_6]_=$	$[x_4]_-$	$[x_1 * x_7]_-$
ALPS	$[x_1]_-$	$[x_3]_-$	$[x_4]_-$	$[x_5]_-$
	$[x_3 * x_4]_=$	$[x_5 * x_6]_=$	$[x_1 * x_2]_=$	$[x_6 * x_{10}]_-$
	$[x_3 * x_6 * x_{10}]_=$	$[x_3 * x_4 + x_5 * x_6]_=$		

$[T]_-$ indicates that the identified subtree T is a subterm of one term in a problem formula, $[T]_{\approx}$ indicates that the identified T is functionally equivalent with one of the terms in a problem formula, $[T]_+$ means T is one term or a sum of two terms in a problem formula, and T means the subtree T are identified by all the three test algorithms

Table 10 Most sampled subtrees for the GP algorithm on the *Aircraft* problem

Subtree infix string	Sample count
<i>Aircraft</i>	
$\exp(x_{18})$	3115
$\sqrt{x_{15}}$	1808
x_{17}^2	1404
$\exp(\sqrt{x_{15}})$	1068
x_{14}^2	824
$[x_5 * x_4]_-$	791
$x_{13} * x_7$	532
$x_6 - x_{16}$	520
$[x_{13} * x_7 * x_{18}]_-$	444
$x_7 + x_7$	353
<i>Friedman-1</i>	
x_1^2	3427
$1 - x_2$	3395
$\exp(x_1^2)$	1772
$1 - x_4$	1715
$1 - x_2 - x_2$	1538
$1 - x_4 - x_2$	1399
$x_2 + x_3$	1004
$1 - x_4 - x_2 - x_4 - x_5$	609
$\sin(1 - x_2 - x_2)$	584
$1 - x_2 - x_4$	529
<i>Friedman-2</i>	
$[x_1 * x_2]_-$	7057
$[\sin(x_1 * x_2)]_{\approx}$	4303
$\sin(x_3)$	2352
$\sin(\sin((x_1 * x_2)))$	688
$\sin(x_1)$	535
$\sin(\sin(x_3))$	502
$[x_3^2]_-$	190
$x_5 + \sin(x_1 * x_2) + x_4$	190
$\sin(x_1 * x_2) + x_4$	176
$\sin(x_4)$	168
<i>GP-Challenge</i>	
$\sqrt{t_7}$	6625
$\cos(t_5)$	3160
$\sin(t_5)$	1904
$\sqrt{t_7} - \cos(t_5)$	561
$\cos(\sqrt{(t_7 - \cos(t_5))})$	445
$t_8 + t_4$	430
$\sin(t_5) + 1$	302
$t_3 - \text{RMSD}$	251
$\cos(\sqrt{t_7})$	247
$\cos(\cos(t_5) - \sqrt{t_7})$	181
<i>Poly-10</i>	
Subtree infix string	
$x_2 * x_1$	1958

Table 10 (continued)

Subtree infix string	Sample count
$x_3 * x_4$	1628
$x_5 * x_6$	1237
$x_9 * x_1$	1138
$x_7 * x_9 * x_1$	983
$\sin(x_5)$	695
x_5/x_6	470
$x_9 + 1$	297
$x_2 * x_1 * (x_9 + 1)$	236
$x_4 * x_6$	187

Table 11 Most sampled subtrees for the OSGP algorithm

Subtree infix string	Sample count
<i>Aircraft</i>	
$\log(x_7)$	18,323
$x_5 * x_4 -$	13,920
$x_{17} * x_{18} -$	12,645
$1/x_{15} -$	12,516
$x_{13} * \log(x_7)$	10,886
$x_{13} * x_7 -$	10,046
$1/x_{15} * x_{17} * x_{18} * x_{14}$	9081
$x_{18} * \log(x_{13} * x_7)$	7863
$x_{18} * \log(x_{13} * x_7)/x_{15} * x_{16} * x_{10}$	7687
$x_{13} * \log(x_7) * \log(x_{13} * \log(x_7))$	6451
<i>Friedman-1</i>	
$1/\exp(x_2)_{\approx}$	24,866
$\exp(x_2)_{\approx}$	23,310
$x_3 + x_2$	14,431
$\exp(x_1^2)$	13,592
$\cos(1/\exp(x_2))$	11,779
$x_4 + 1$	11,398
$\cos(1/\exp(x_2) - \cos(1/\exp(x_2)))$	11,005
$x_2 - \cos(1/\exp(x_2) - \cos(1/\exp(x_2))) + x_3 + x_4 + 1$	9294
x_1^2	8669
$x_5 + 1$	8515
<i>Friedman-2</i>	
$[x_1 * x_2]_-$	18,910
$\sin(\sin(x_3))$	14,557
$x_4 + \sin(x_1 * x_2)$	12,966
$x_5 + 1$	11,337
$\sin(x_3)$	9690
$1 - \sin(\sin(x_3))$	9651
$x_1 * x_2 * \sin(x_3)$	9216
$1 - x_1 * x_2 * \sin(x_3)/2$	5749

Table 11 (continued)

Subtree infix string	Sample count
$x_3/(1 - x_1 * x_2 * \sin(x_3)/2)$	5710
$\sin(1) + \sin(1) + x_3/(1 - x_1 * x_2 * \sin(x_3)/2)$	4060
<i>GP-Challenge</i>	
$\sqrt{t_7}$	28567
$\sqrt{t_7} - (1 - \text{RMSD})$	12,261
$\sin(\text{RMSD})$	12,219
$\sin(t_5) - (\text{RMSD} + t_7)$	12,099
$\log(1) - \sin(t_5)$	11,699
$(\text{RMSD} - (\sqrt{t_7} - (1 - \text{RMSD})))^2$	11,485
$\sin(\sqrt{t_7} + \sin(t_5) - (\text{RMSD} + t_7))$	11,342
$t_8 - t_3 + t_7$	11,097
$\sin(t_5)$	10178
$\log(\sqrt{t_7}) - \sin(t_5)$	9540
<i>Poly-10</i>	
$x_6 * x_5$	17,457
$x_3 - x_4$	15,946
$x_3 * x_4$	15,120
$[x_4]_-$	14,856
$[x_7 * x_1]_-$	14073
$x_5 - x_1$	13,376
$\exp(x_6 * (x_3 - x_4))$	12,697
$\sin(x_2) * (x_7 * x_1 - x_1)$	12,381
$x_9 * (x_5 - x_1) - x_6 * x_1$	12,144
$\sin(x_2) * (x_7 * x_1 - x_1) - x_3 * x_4 + \exp(x_6 * (x_3 - x_4)) * x_4$	12,058

Table 12 Most sampled subtrees for the ALPS algorithm on the *Aircraft* problem

Subtree infix string	Sample count
<i>Layer 0</i>	
$[x_5]_-$	580
$x_{18} + 1$	572
$\sqrt{x_2}$	487
$\exp(x_5)$	420
$\sqrt{x_2} + x_5$	360
$x_5/\sqrt{x_2} + x_7$	214
$x_{18} + x_7$	200
$\log(\exp(x_5))$	187
$x_{18} + x_4 * (\sqrt{x_2} + x_5)$	162
$x_4 * (x_{17} + x_{18})$	150
<i>Layer 1</i>	
$x_{10} + x_{18}^2$	310
$x_{10} + x_{18}^2 - x_{18}$	287
$x_5 + \sin(1)$	230
$(x_{16} * x_4 * (x_5 + \sin(1)) + 1)/x_{16}$	185
x_{18}^2	119

Table 12 (continued)

Subtree infix string	Sample count
$[x_{10}]_-$	115
$(x_{10} + x_{18}^2 - x_{18} + 3)/x_{15}$	104
$1/x_{16}$	101
$x_{13} - 1$	94
$1/x_{18}$	86
<i>Layer 2</i>	
$[x_4 * x_5]_-$	287
$x_{13} - 1$	279
$x_{10} + x_{18}^2$	179
x_{18}^2	165
$[x_{10}]_-$	155
$x_{10} + x_{18}^2$	124
$x_{10} + x_{13} + x_{18}^2$	92
$x_{13} - \sqrt{x_2}$	80
$(x_{10} + x_{13} + x_{18}^2 + x_4 * x_5)/x_{15}$	76
$-x_{13} + \sqrt{x_2} + x_4 * x_5 + 1/x_{16}$	74
<i>Layer 3</i>	
$x_{13} - 1$	443
$[x_4 * x_5]_-$	409
$x_{10} + x_{18}^2$	222
$x_{10} + x_{13} + x_{18}^2$	203
$\sqrt{x_7}$	172
$-x_{13} + x_4 * x_5 + 1 + 1/x_{16}$	164
$(x_{10} + x_{13} + x_{18}^2 + x_4 * x_5)/x_{15}$	88
$[x_{10}]_-$	55
$1/x_{16}$	47
$(x_{15} * x_{16} * (-x_{13} + x_4 * x_5 - \sqrt{x_7} + 1) + x_{15})$	
$-x_{16} * (x_{10} + x_{13} + x_{18}^2 + x_4 * x_5)/(x_{15} * x_{16})$	44
<i>Layer 4</i>	
$x_{10} + x_7$	423
$x_4 * x_5$	335
$x_{16}^{3/2} * (x_{10} + x_7)$	291
$-x_{15} + x_{18} + x_4 * x_5$	277
$\sqrt{x_{14}}$	189
$x_4 + x_7 + 1$	170
$x_{14} * x_{17}$	166
$-(x_{10} + x_7) * (-x_{18} + x_4 + x_7 + 1)$	145
$x_{14} * x_{17} - x_4 * x_5 + x_7$	140
$x_{13} * (x_{10} + x_7) * (x_{16}^{3/2} - x_{18} + x_4 + x_7 + 1)$	125
<i>Layer 5</i>	
$x_4 * x_5_-$	280
$x_{10} + x_7$	260
$-x_{15} + x_{18} + x_4 * x_5$	223
$x_{16}^{3/2} * (x_{10} + x_7)$	207
$x_{14} * x_{17}$	136
$\sqrt{x_{14}}$	122
$x_{14} * x_{17} - x_4 * x_5 + x_7$	119

Table 12 (continued)

Subtree infix string	Sample count
$x_{10} + \sqrt{x_{14}} + x_{17}$	96
$x_4 + x_7 + 1$	93
$(-x_{14} + x_{17})^2$	79

Table 13 Most sampled subtrees for the ALPS algorithm on the *Friedman-I* problem

Subtree infix string	Sample count
<i>Layer 0</i>	
$\sin(x_2)$	1115
$\cos(\sqrt{x_3})$	506
x_1^2	338
$\sqrt{x_3}$	260
x_2^2	124
$x_1 * x_2 + \sin(x_2)$	106
$\sin(x_2) + \cos(x_2^2)$	103
$-\sin(x_2) + \cos(1)$	103
$[x_3]_-$	93
$\sin(\cos(x_2))$	82
<i>Layer 1</i>	
$\sin(x_2)$	352
$[\exp(2 * x_1)]_{\approx}$	319
$\exp(x_3) + \cos(x_1)$	276
$x_2 + \exp(x_3) + \cos(x_1) + 1$	251
$-2 * x_1 + x_4$	240
$2 * x_1$	230
$2 * x_2 + \exp(2 * x_1) + \exp(x_3) + \cos(x_1)$	186
$\exp(x_3)$	104
$\cos(x_1)$	100
$\exp(2 * x_1) - 1$	99
<i>Layer 2</i>	
$x_1 + 1$	224
$\sin(x_2)$	198
$\cos(\cos(x_1 + 1))$	195
$x_1 - x_3$	134
$2 * x_1 - 1$	117
$\exp(x_1 - x_3 - x_5)$	92
$(-x_1 + x_3) * \exp(x_1 - x_3 - x_5)$	
$-\cos(\cos(x_1 + 1)))$	72
$x_2 + \cos(1)$	70
$x_2 + \log(2 * x_1 - 1)$	69
$\sin(x_2 + \log(2 * x_1 - 1) + \sin(x_2))$	66
<i>Layer 3</i>	
x_1^2	431
$1 - x_2$	210
$\sqrt{x_3}$	201

Table 13 (continued)

Subtree infix string	Sample count
$x_2 + x_3$	200
$-\sin(x_2 - 1)$	194
$x_4 + (-\sin(x_2 - 1))^2$	188
$\cos(\sin(x_1^2))$	186
$x_5 * \sin(1)$	173
$\sqrt{x_3} + (x_2 + x_3) *$	
$x_4 + (-\sin(x_2 - 1))^2 + x_1^2$	172
$\sin(\cos(\sin(x_1^2)))$	159
<i>Layer 4</i>	
x_1^2	609
$\sin(x_2^2)$	408
$x_2 + 1$	300
$x_2 + x_3$	255
$(\exp(x_5))^2$	248
$2 * x_1^2$	179
$(2 * x_1 * x_1^2)^2$	171
$\sin(x_2 + \sin(x_2^2) + 1)$	167
$x_2 + x_3 + x_4 + (\exp(x_5))^2 + \sin(x_2 + \sin(x_2^2) + 1)$	161
x_2^2	152
<i>Layer 5</i>	
x_1^2	474
$2 * x_2$	368
$\sin(x_2^2)$	143
$\sin(x_1)$	140
$\sin(x_5)$	136
$\sin(3 * x_2)$	123
$x_1 * (x_1^2 + \sin(x_1))$	120
$x_2 + x_4 + \cos(\sin(3 * x_2))$	120
$(2 * x_2 - 1) * \sin(x_2)$	113
$x_2 * \sin((2 * x_2 - 1) * \sin(x_2))$	110

Table 14 Most sampled subtrees for the ALPS algorithm on the *Friedman-II* problem

Subtree infix string	Sample count
<i>Layer 0</i>	
$\sin(x_1)$	1452
$x_4 + \sin(x_1)$	562
$x_2 - 1$	269
$\sin(x_2 - 1)$	265
$\sqrt{\sin(x_1)}$	216
$x_2 + \sin(x_1) + 2$	193
$\sin(x_2)$	161
$\sin(x_1) - 1$	140
$x_2 + 1$	126
$2 * x_4 + 2 * \sin(x_1) - 1$	108

Table 14 (continued)

Subtree infix string	Sample count
<i>Layer 1</i>	
$\sin(x_1)$	628
$x_2 + x_4$	251
$x_4 + x_5$	245
$-x_1 + x_2$	214
$x_4 + 2 * \sin(x_1)$	168
$x_2 + \sqrt{\sin(x_1)}$	158
$-x_4$	156
$-x_1 - x_2 - x_4 + x_5$	152
$x_1 - x_4$	103
$x_4 + \sin(x_1)$	98
<i>Layer 2</i>	
$x_2 + x_4$	127
$x_4 + x_5$	125
$-x_1 + x_2$	111
$x_2 + \sqrt{\sin(x_1)}$	78
$-x_4$	78
$x_4 + 2 * \sin(x_1)$	71
$-x_1 - x_2 - x_4 + x_5$	71
$\sqrt{\sin(x_1)}$	67
$x_1 - x_4$	66
$\sin(x_2)$	64
<i>Layer 3</i>	
$[x_2]_-$	319
$\sqrt{x_2}$	235
$x_1 * \sqrt{x_2}$	231
$\cos(x_1)$	202
$\sqrt{x_2} + \sin(x_2)$	181
$x_1 * (\sqrt{x_2} + \sin(x_2)) * \sin(x_2)$	177
$x_2 + \sin(\cos(x_1))$	168
$x_1 * \sin(x_1 * (\sqrt{x_2} + \sin(x_2)) * \sin(x_2))$	166
$\sqrt{x_1} + x_5$	156
$x_2 + x_4 + x_5 + \sin(\cos(x_1))$	154
<i>Layer 4</i>	
$x_1 * x_2_-$	274
x_5_-	245
$\cos(\sqrt{x_1 * x_2})$	234
$\exp(\cos(\cos(\sqrt{x_1 * x_2})))$	213
$(x_4 + \exp(\cos(\cos(\sqrt{x_1 * x_2})))) * \exp(x_5)$	205
$\exp(x_5)$	189
$\sin(x_3^2)$	182
$[x_3^2]_-$	168
$\exp(x_3^2)$	146
$\sin(\sin(\sin(x_3^2)))$	138
<i>Layer 5</i>	
$\cos(x_3)$	676
$\sin(x_3)$	529

Table 14 (continued)

Subtree infix string	Sample count
$\cos(\sin(x_3))$	451
$[x_1 * x_2]_-$	379
$\sin(x_3) + 1$	359
$x_3 + \cos(1)$	308
$\sin(x_1 * x_2) + \cos(\sin(x_3) + 1)$	207
$\sin(x_1 * x_2)_-$	124
x_4^2	113
$x_3 + \sin(x_1) - \cos(x_3) + \cos(1)$	88

Table 15 Most sampled subtrees for the ALPS algorithm on the *GP-Challenge* problem

Subtree infix string	Sample count
<i>Layer 0</i>	
$\log(\text{RMSD})$	878
RMSD	304
$\log(\log(\text{RMSD}))$	295
$t_7 + \log(\log(\text{RMSD}))$	124
$\exp(\text{RMSD})$	120
$\log(\text{RMSD})/\text{RMSD}$	117
$\exp(e/2) * \log(\log(\text{RMSD}))$	99
$\log(\log(\text{RMSD}))$	93
$\sqrt{\log(\log(\text{RMSD}))}$	75
$\sqrt{\exp(e/2) * \log(\log(\text{RMSD})) + e}$	74
<i>Layer 1</i>	
$\log(t_7 + t_8)$	343
$\log(\text{RMSD})$	288
$\text{RMSD} * \cos(\log(t_7 + t_8))$	269
$\text{RMSD}^2 * \cos(\log(t_7 + t_8)) - t_7$	254
$t_7 + t_8$	184
$\log(\log(\text{RMSD}))$	112
$\cos(\text{rank})$	110
$\exp(\cos(\text{rank}))$	102
$\cos(\log(t_7 + t_8))$	73
$\sin(\text{RMSD})$	62
<i>Layer 2</i>	
$\sin(t_7) - 1$	387
$\sqrt{\exp(\text{RMSD})}$	349
$(\sin(t_7)^2 - 1) + 1$	319
$(-(\sin(t_7) - 1)^2 + \sqrt{\exp(\text{RMSD})} - 1)^2$	287
$\sin(t_7)$	205
$(-(\sin(t_7) - 1)^2 + \sqrt{\exp(\text{RMSD})} - 1)^2 + \log(t_8) - 1$	200
$\exp(\text{RMSD})$	97
$(\sin(t_7) - 1)^2$	90

Table 15 (continued)

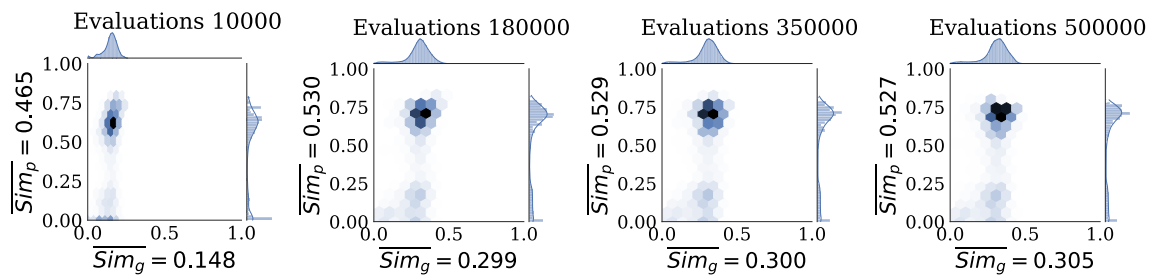
Subtree infix string	Sample count
t_1^2	77
$\sin(t_4)$	77
Layer 3	
$\sqrt{t_7}$	445
$\log(t_7)$	343
$\text{RMSD} + t_7$	304
$t_3 - 1$	304
$\cos((\text{RMSD} + t_7)/\text{RMSD})$	290
$-\cos(\sqrt{t_7}) - \cos((\text{RMSD} + t_7)/\text{RMSD}) + 1$	269
$\text{RMSD} * \log(t_7) + t_3 - t_8 - 1$	217
$t_4 + 1$	204
$-t_4 + t_7$	134
$\sqrt{-t_4 + 2 * t_7}$	128
Layer 4	
$\sqrt{t_7}$	353
$t_3 - 1$	228
$-\text{RMSD} + t_7$	213
$(\cos(\sqrt{t_7}))^2$	194
$\sqrt{-\text{RMSD} + t_7} - \exp((\cos(\sqrt{t_7}))^2)$	185
$-\text{RMSD} + t_3 - t_8$	169
$-\text{RMSD} - t_3 - t_8 + 1$	161
$(-t_3 + 1)^2$	159
$(-\text{RMSD} - t_3 - t_8 + 1)^2 * \exp(t_1)$	152
$\text{RMSD} - \sqrt{t_7}$	134
Layer 5	
$\cos(t_5)$	223
$\log(2 * t_7)$	203
$-t_4 * \cos(t_5) + t_8$	193
$\sin(\sin(\log(2 * t_7)))$	188
$-t_4 + t_8$	182
$t_3 * \sin(\log(t_3 * \sin(\sin(\log(2 * t_7)))))$	181
$\log(t_3 * \sin(\sin(\log(2 * t_7))))$	181
$\text{RMSD} - t_4 * \cos(t_5) - t_4 + 2 * t_8$	159
$-2 * \text{RMSD} + t_7 + 1$	133
$2 * \text{RMSD} - t_4 * \cos(t_5) - t_4 + 2 * t_8 + 1$	126

Table 16 Most sampled subtrees for the ALPS algorithm on the *Poly-10* problem

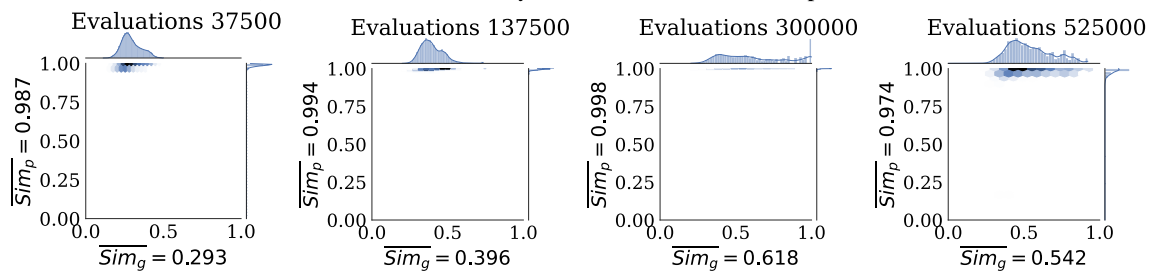
Subtree infix string	Sample count
Layer 0	
$x_5 * \sin(1)$	972
$\sqrt{\exp(x_6)}$	742
$\exp(x_{10} + x_4)$	421
$[x_5]_-$	389
$\cos(\sqrt{\exp(x_{10} + x_4)})$	389
$x_6 / (x_5 * \sqrt{\exp(x_6)} * \sin(1))$	355
$\log(x_6 * \cos(\sqrt{\exp(x_{10} + x_4)}) / (x_5 * \sqrt{\exp(x_6)} * \sin(1)))$	334
$\exp(x_6)$	300
$(x_5 * \sqrt{\exp(x_6)} * \sin(1))^2$	265
$(\log((x_5 * \sqrt{\exp(x_6)} * \sin(1))^2))^2$	245
Layer 1	
$[x_4]_-$	495
x_3	358
$1 - x_5$	256
$x_3 * x_4 + x_5 * x_6$	251
$\cos(x_{10})$	190
$x_5 * x_6$	169
$x_3 * x_4$	101
$(1 - \cos(x_{10})) / (x_5 - 2)$	88
$x_6 * (x_5 + \sin(1))$	82
$x_4 * \sin(1) + x_5 * x_6$	69
Layer 2	
$[x_4]_-$	227
$\cos(x_{10})$	189
$[x_3]_-$	143
$x_3 * x_4 + x_5 * x_6$	115
$1 - x_5$	111
$x_6 * \exp(-x_5)$	90
$\exp(x_5)$	84
$\cos(\cos(x_{10}))$	61
$x_6 * (x_5 + \sin(1))$	56
$1 / (\cos(\cos(x_{10})) + 1)$	55
Layer 3	
$x_5 * x_6$	378
$[x_1]_-$	337
$x_3 * x_4$	214
$x_6 * \exp(-x_5)$	137
$x_1 * x_2$	126
$[x_4]_-$	110
$x_4 - x_5 * x_6$	90
$-x_5 * x_6 - x_6 * \exp(-x_5) + 1$	83
$\exp(x_5)$	77
$x_3 * x_4 + x_5 * x_6 + x_6 * \exp(-x_5) - 1$	73
Layer 4	
$[x_{10} * x_6]_-$	390

Table 16 (continued)

Subtree infix string	Sample count
$x_5 * x_6$	345
$\sin(x_4)$	261
$x_5 + x_6$	234
$x_1 * \sin(x_2)$	232
$x_3 * \sin(x_4)$	221
$x_5 + x_6 + 2 * x_9$	189
$\cos(x_{10} * x_6 + x_3)$	187
$x_{10} * x_3 * x_6$	179
$x_1 * x_7 * (x_5 + x_6 + 2 * x_9)$	176
<i>Layer 5</i>	
$x_{10} * x_6$	359
$\sin(x_4)$	236
$x_3 * \sin(x_4)$	208
$\sin(x_2)$	196
$x_{10} * x_6 + x_3$	189
$x_5 * x_6 - \cos(x_{10} * x_6 + x_3)$	178
$\cos(x_{10} * x_6)$	140
$x_5 * x_6$	105
$x_{10} * x_3 * x_6$	94
$\cos(x_{10} * x_6 + x_3)$	88

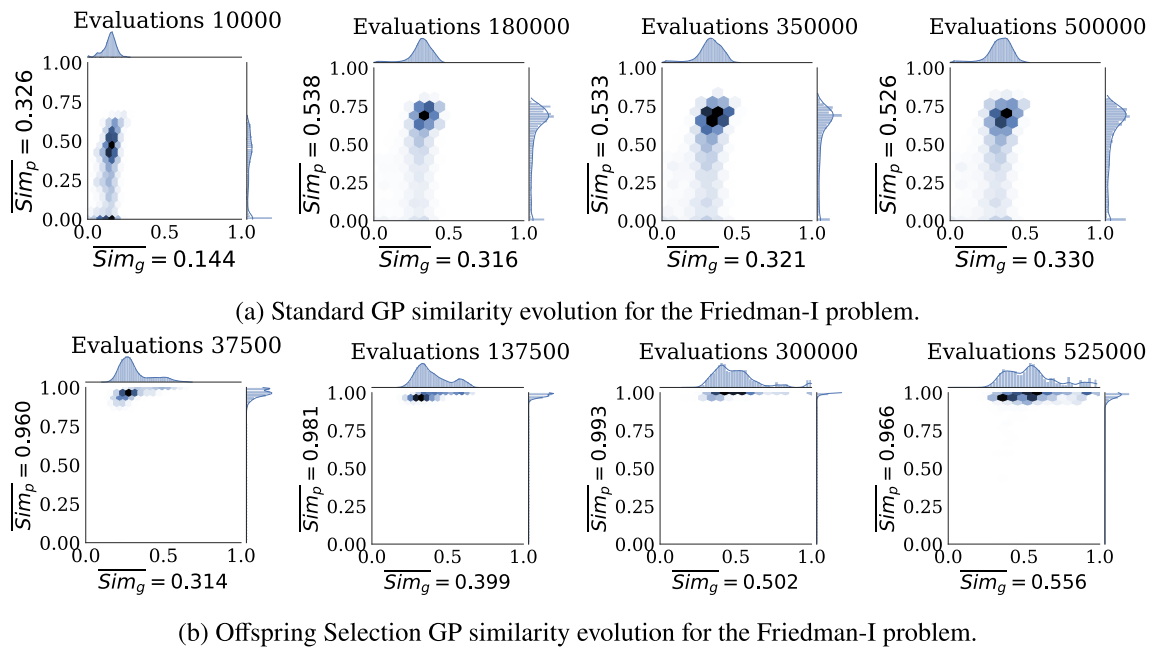
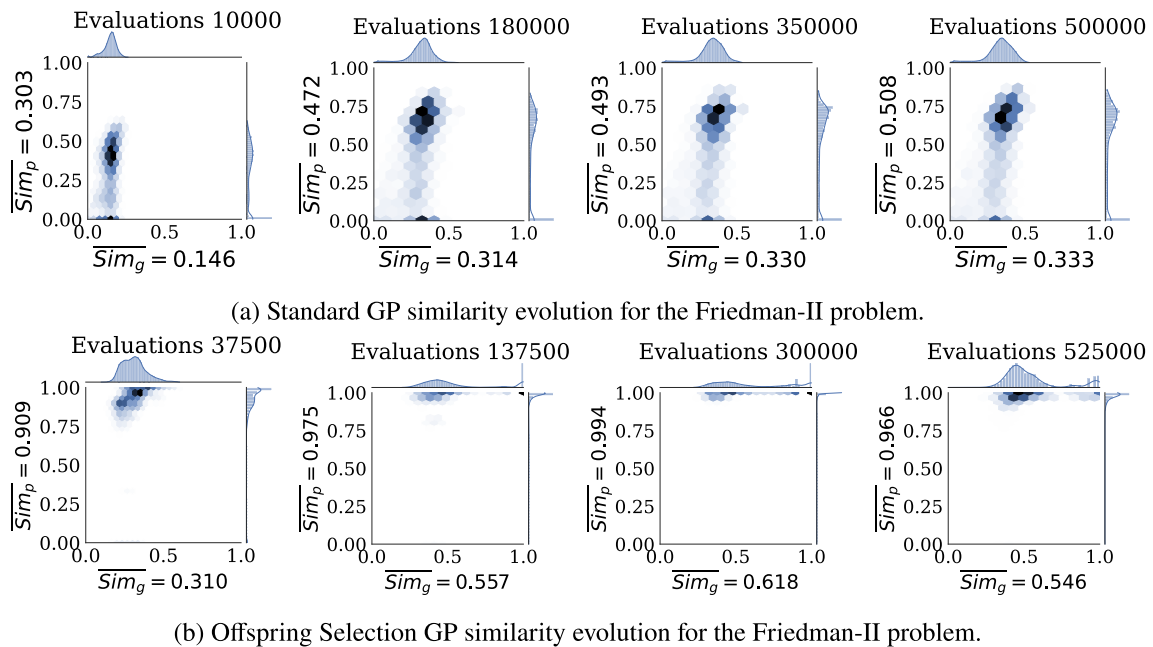


(a) Standard GP similarity evolution for the Aircraft problem.



(b) Offspring Selection GP similarity evolution for the Aircraft problem.

Fig. 6 Similarity evolution on the *Aircraft* problem

Fig. 7 Similarity evolution on the *Friedman-I* problemFig. 8 Similarity evolution on the *Friedman-II* problem

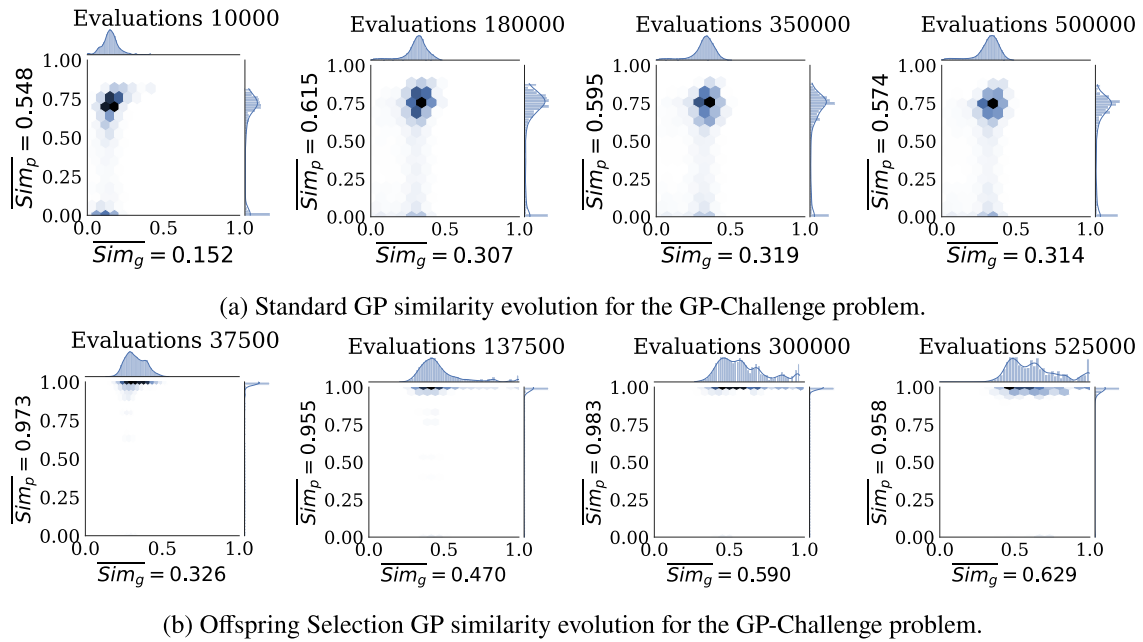


Fig. 9 Similarity evolution on the *GP-Challenge* problem

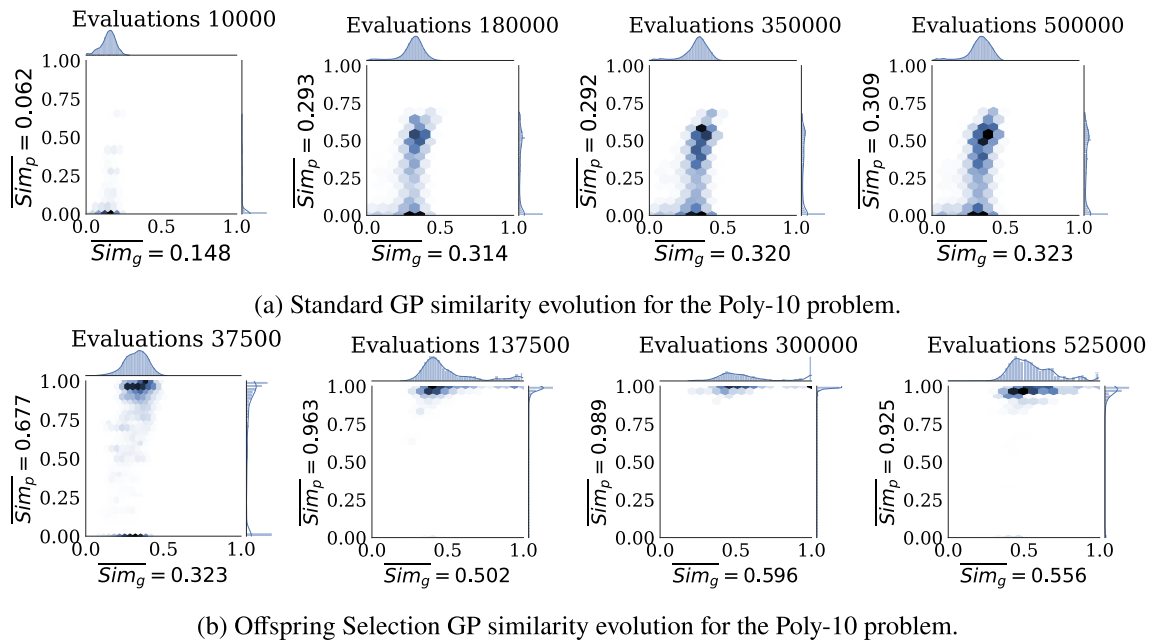
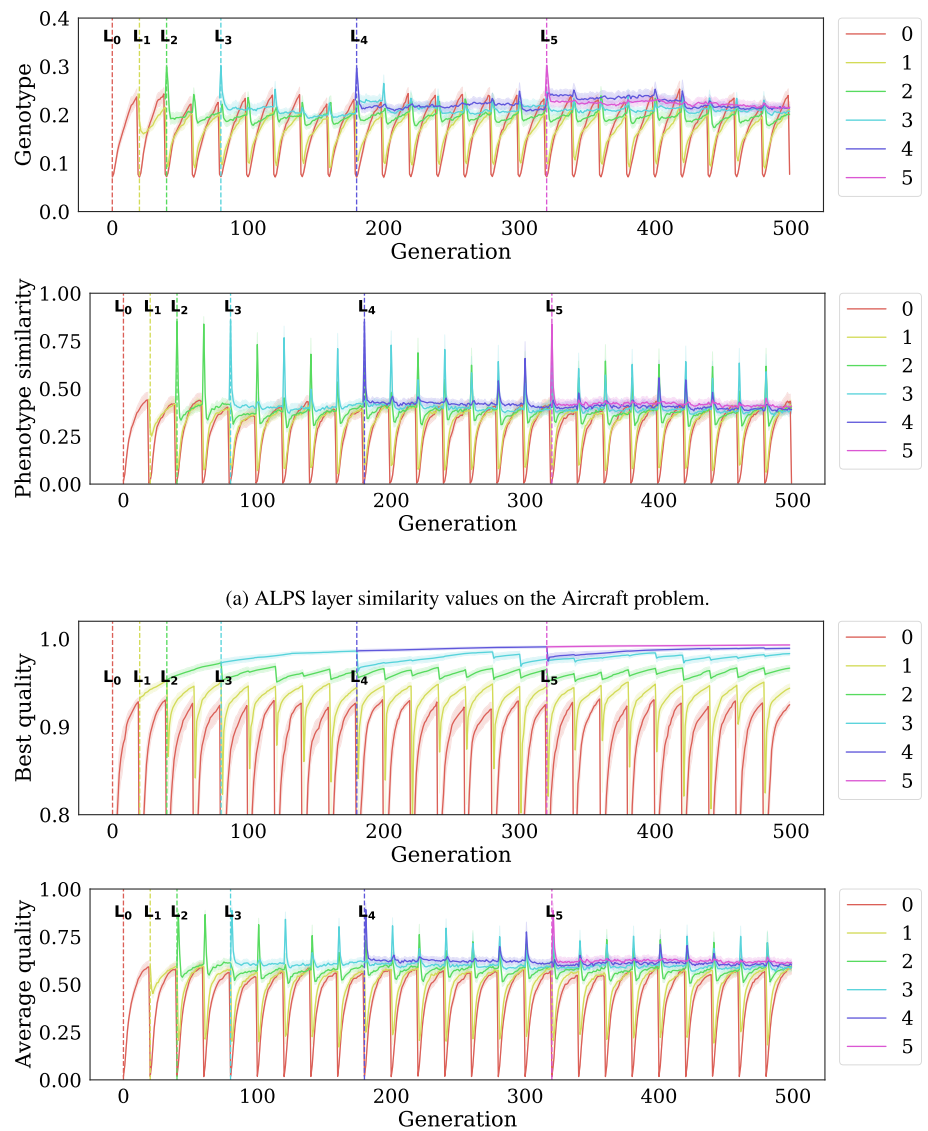


Fig. 10 Similarity evolution on the *Poly-10* problem

Fig. 11 Dynamic analysis of the ALPS on the *Aircraft* problem



(b) ALPS layer best and average quality values on the Aircraft problem.

Fig. 12 Dynamic analysis of the ALPS on the *Friedman-I* problem

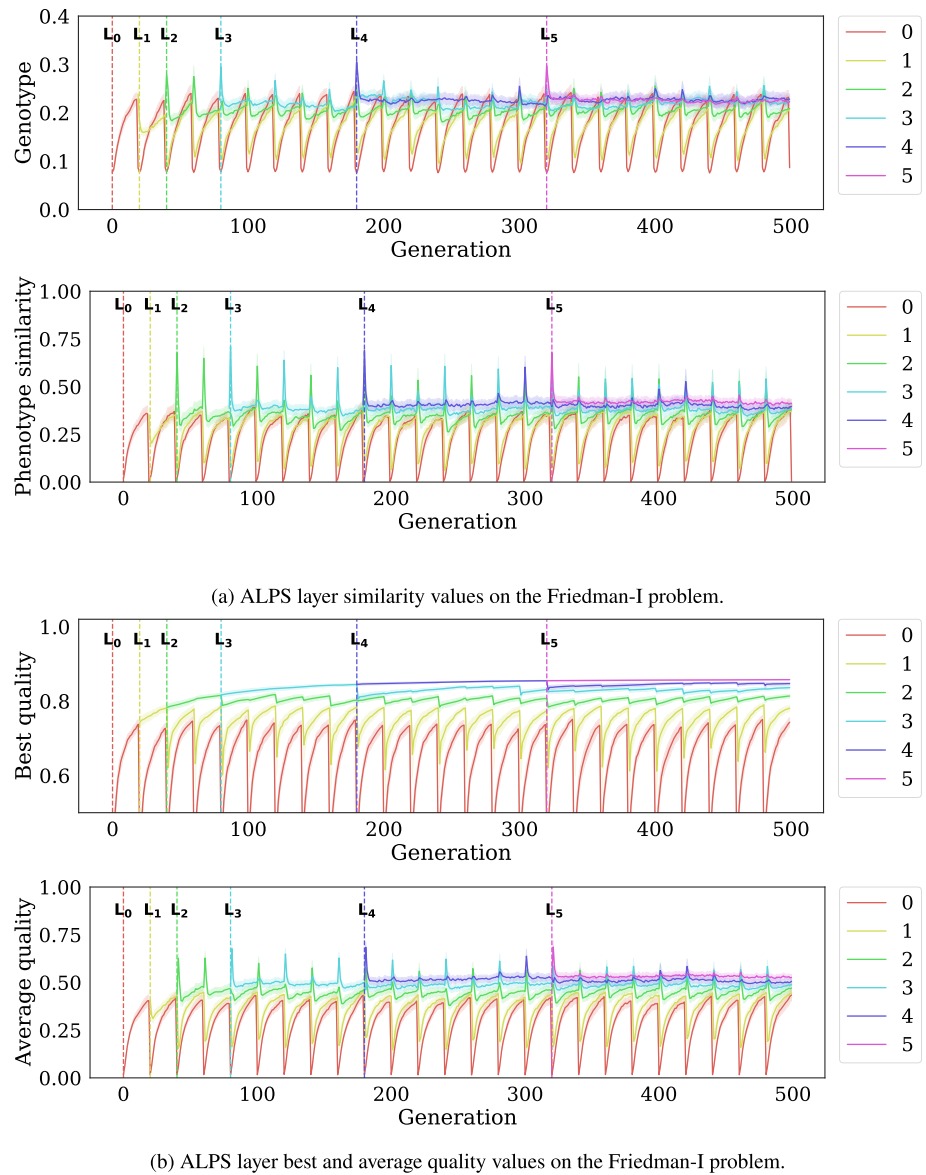


Fig. 13 Dynamic analysis of the ALPS on the *Friedman-II* problem

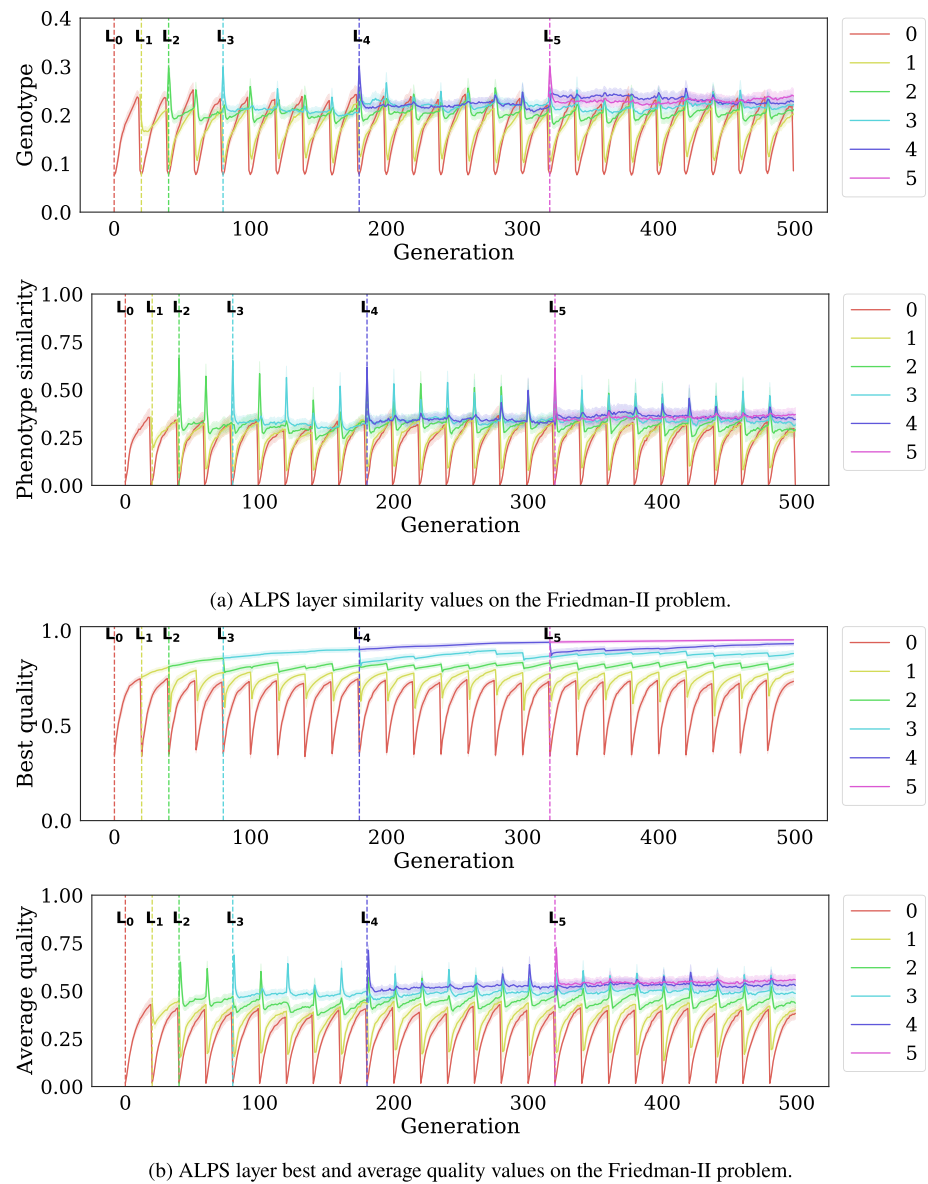


Fig. 14 Dynamic analysis of the ALPS on the *GP-Challenge* problem

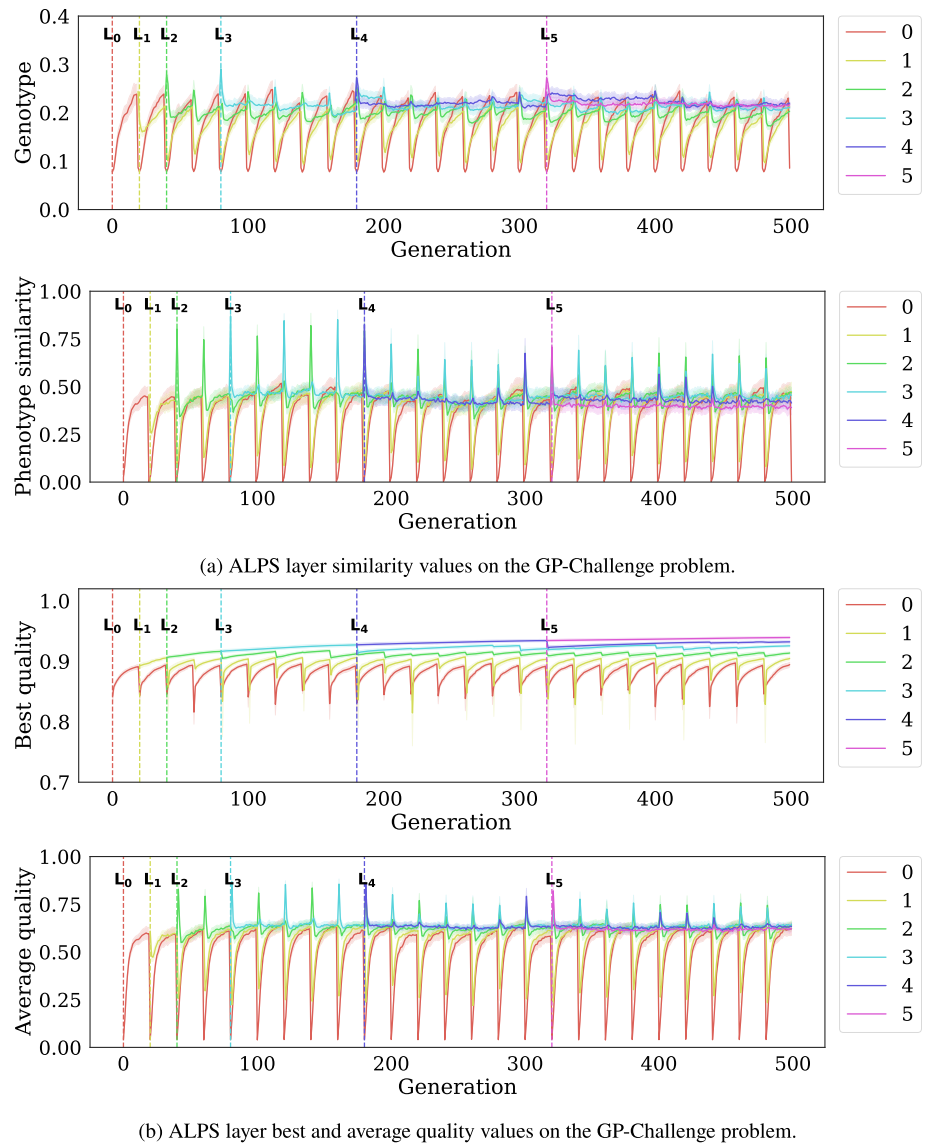
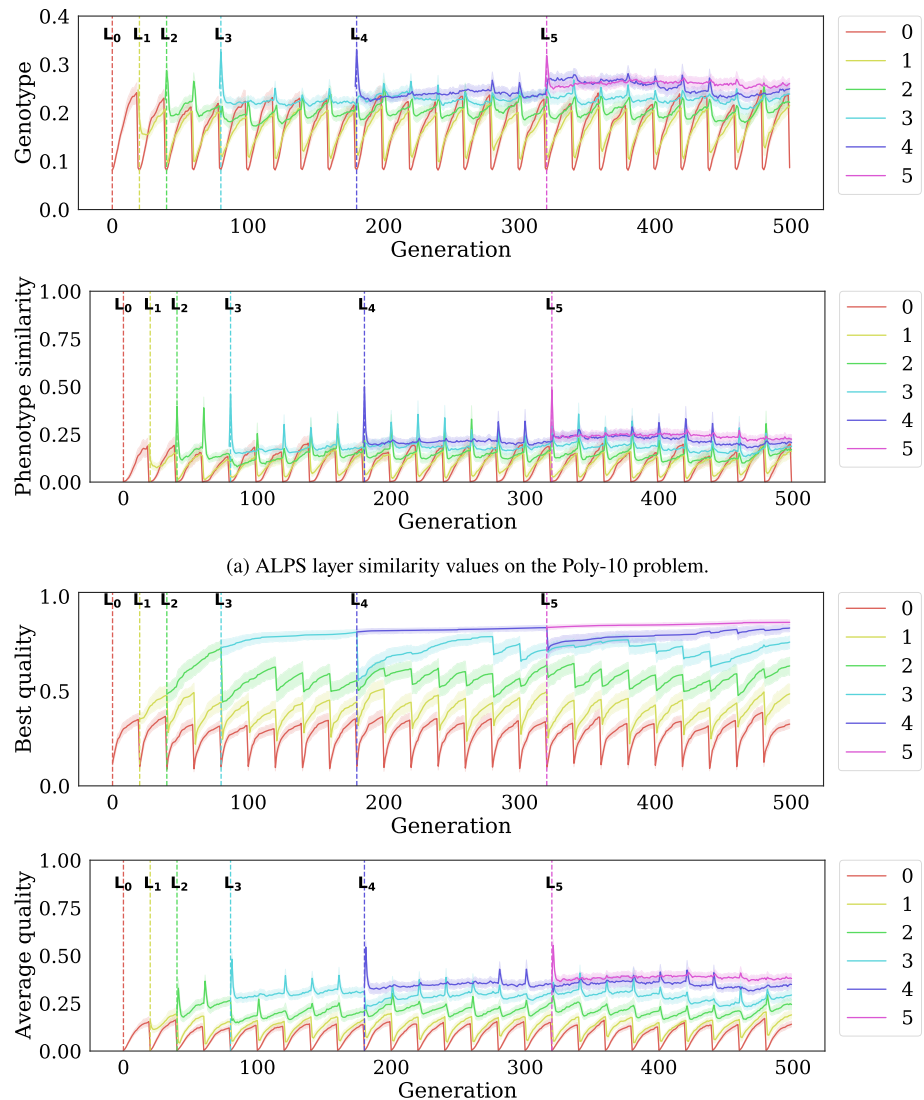


Fig. 15 Dynamic analysis of the ALPS on the *Poly-10* problem



(a) ALPS layer similarity values on the Poly-10 problem.

(b) ALPS layer best and average quality values on the Poly-10 problem.

Funding Open access funding provided by University of Applied Sciences Upper Austria.

Open Access This article is licensed under a Creative Commons Attribution 4.0 International License, which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if changes were made. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit <http://creativecommons.org/licenses/by/4.0/>.

References

- Affenzeller M, Wagner S, Winkler S (2005) GA-selection revisited from an ES-driven point of view. In: Mira J, Alvarez JR (eds) Artificial intelligence and knowledge engineering applications: a bioinspired approach, lecture notes in computer science, vol 3562. Springer, Berlin, pp 262–271
- Affenzeller M, Winkler S, Wagner S, Beham A (2009) Genetic algorithms and genetic programming: modern concepts and practical applications, 1st edn. Chapman & Hall/CRC, London
- Affenzeller M, Winkler SM, Burlacu B, Kronberger G, Kommenda M, Wagner S (2017) Dynamic observation of genotypic and phenotypic diversity for different symbolic regression GP variants. In: Proceedings of the genetic and evolutionary computation conference companion, pp 1553–1558
- Affenzeller M, Winkler SM, Kronberger G, Kommenda M, Burlacu B, Wagner S (2014) Gaining deeper insights in symbolic regression. In: Genetic programming theory and practice XI. Springer, pp 175–190

- Agapitos A, Loughran R, Nicolau M, Lucas S, O'Neill M, Brabazon A (2019) A survey of statistical machine learning elements in genetic programming. *IEEE Trans Evol Comput* 23(6):1029–1048
- Bessaou M, Pétrowski A, Siarry P (2000) Island model cooperating with speciation for multimodal optimization. In: International conference on parallel problem solving from nature. Springer, pp 437–446
- Bille P (2005) A survey on tree edit distance and related problems. *Theor Comput Sci* 337(1):217–239. <https://doi.org/10.1016/j.tcs.2004.12.030>
- Burke EK, Gustafson S, Kendall G (2004) Diversity in genetic programming: an analysis of measures and correlation with fitness. *IEEE Trans Evol Comput* 8(1):47–62
- Burke E, Gustafson S, Kendall G, Krasnogor N (2002) Advanced population diversity measures in genetic programming. In: International conference on parallel problem solving from nature. Springer, pp 341–350
- Burks AR, Punch WF (2015) An efficient structural diversity technique for genetic programming. In: Proceedings of the 2015 annual conference on genetic and evolutionary computation, pp 991–998
- Burks AR, Punch WF (2017) An analysis of the genetic marker diversity algorithm for genetic programming. *Genet Program Evol Mach* 18(2):213–245
- Burks AR, Punch WF (2018) An investigation of hybrid structural and behavioral diversity methods in genetic programming. Springer, Cham, pp 19–34. https://doi.org/10.1007/978-3-319-97088-2_2
- Burlacu B, Affenzeller M, Winkler S, Kommenda M, Kronberger G (2015) Methods for genealogy and building block analysis in genetic programming. Springer, Cham, pp 61–74. https://doi.org/10.1007/978-3-319-15720-7_5
- Burlacu B, Affenzeller M, Kronberger G, Kommenda M (2019) Online diversity control in symbolic regression via a fast hash-based tree similarity measure. In: 2019 IEEE congress on evolutionary computation (CEC), pp 2175–2182
- Burlacu B, Kommenda M, Affenzeller M (2015) Building blocks identification based on subtree sample counts for genetic programming. In: 2015 Asia-Pacific conference on computer aided system engineering. IEEE, pp 152–157
- Chen C, Luo C, Jiang Z (2018) A multilevel block building algorithm for fast modeling generalized separable systems. *Expert Syst Appl* 109:25–34. <https://doi.org/10.1016/j.eswa.2018.05.021>
- Ciesielski V, Li X (2007) Data mining of genetic programming run logs. In: Ebner M, O'Neill M, Ekárt A, Vanneschi L, Esparcia-Alcázar AI (eds) Genetic programming. Springer, Berlin, pp 281–290
- Cramer NL (1985) A representation for the adaptive generation of simple sequential programs. In: Proceedings of an international conference on genetic algorithms and the applications, pp 183–187
- Črepinšek M, Liu SH, Mernik M (2013) Exploration and exploitation in evolutionary algorithms: a survey. *ACM Comput Surv.* <https://doi.org/10.1145/2480741.2480752>
- De Jong KA (1975) An analysis of the behavior of a class of genetic adaptive systems. Ph.D. thesis, USA. AAI7609381
- De Jong KA (1975) Analysis of the behavior of a class of genetic adaptive systems. Technical report
- De Jong K (2007) Parameter setting in EAs: a 30 year perspective. Springer, Berlin, pp 1–18. https://doi.org/10.1007/978-3-540-69432-8_1
- de Jong ED, Watson RA, Pollack JB (2001) Reducing bloat and promoting diversity using multi-objective methods. In: Proceedings of the 3rd annual conference on genetic and evolutionary computation, GECCO'01. Morgan Kaufmann Publishers Inc., San Francisco, CA, USA, pp 11–18
- D'haeseleer P, Blum J (1994) Advances in genetic programming. Kinnear, Jr.[451], pp 177–198
- Donatucci D, Dramdahl MK, McPhee N, Morris M (2014) Analysis of genetic programming ancestry using a graph database. In: Midwest instruction and computing symposium
- Eiben AE, Schippers CA (1998) On evolutionary exploration and exploitation. *Fund Inform* 35(1–4):35–50
- Eiben AE, Michalewicz Z, Schoenauer M, Smith JE (2007) Parameter control in evolutionary algorithms. Springer, Berlin, pp 19–46. https://doi.org/10.1007/978-3-540-69432-8_2
- Ekárt A, Gustafson S (2004) A data structure for improved gp analysis via efficient computation and visualisation of population measures. In: European conference on genetic programming. Springer, pp 35–46
- Ekárt A, Németh SZ (2000) A metric for genetic programs and fitness sharing. In: European conference on genetic programming. Springer, pp 259–270
- Eshelman LJ, Schaffer JD (1993) Crossover's niche. In: Proceedings of the 5th international conference on genetic algorithms. Morgan Kaufmann Publishers Inc., San Francisco, pp 9–14
- Fleck P (2015) Age-layered population structure genetic algorithm: behavior analysis and performance comparison. Master's thesis, University of Applied Sciences Upper Austria
- Forsyth R et al (1981) Beagle—a Darwinian approach to pattern recognition. *Kybernetes* 10(3):159–166
- Friedman JH (1991) Multivariate adaptive regression splines. *Ann Stat* 19:1–67
- Goldberg DE, Richardson J, et al (1987) Genetic algorithms with sharing for multimodal function optimization. In: Genetic algorithms and their applications: proceedings of the second international conference on genetic algorithms. Lawrence Erlbaum, Hillsdale, pp 41–49
- Harik GR (1995) Finding multimodal solutions using restricted tournament selection. In: Proceedings of the 6th international conference on genetic algorithms. Morgan Kaufmann Publishers Inc., San Francisco, CA, USA, pp 24–31
- Hicklin JF (1986) Application of the genetic algorithm to automatic program generation. Ph.D. thesis, University of Idaho
- Holland JH (1975) Adaptation in natural and artificial systems. The University of Michigan Press, Michigan
- Holmes P, Barclay PJ (1996) Functional languages on linear chromosomes. In: Proceedings of the 1st annual conference on genetic programming. MIT Press, Cambridge, MA, USA, p 427
- Hornby GS (2006) ALPS: the age-layered population structure for reducing the problem of premature convergence. In: Proceedings of the 8th annual conference on genetic and evolutionary computation, GECCO '06, p. 815–822. Association for Computing Machinery, New York, NY, USA. <https://doi.org/10.1145/1143997.1144142>
- Hu JJ, Goodman ED (2002) The hierarchical fair competition (hfc) model for parallel evolutionary algorithms. In: Proceedings of the 2002 congress on evolutionary computation. CEC'02 (Cat. No. 02TH8600), vol 1, pp 49–54
- Jackson D (2010) The identification and exploitation of dormancy in genetic programming. *Genet Program Evol Mach* 11(1):89–121. <https://doi.org/10.1007/s10710-009-9086-1>
- Joó A (2010) Towards identifying salient patterns in genetic programming individuals. Ph.D. thesis, Aston University
- Kameya Y, Kumagai J, Kurata Y (2008) Accelerating genetic programming by frequent subtree mining. In: Proceedings of the 10th annual conference on genetic and evolutionary computation, GECCO '08. Association for Computing Machinery, New York, NY, USA, pp 1203–1210. <https://doi.org/10.1145/1389095.1389332>

- Keijzer M (2003) Improving symbolic regression with interval arithmetic and linear scaling. In: Ryan C, Soule T, Keijzer M, Tsang E, Poli R, Costa E (eds) Genetic programming, proceedings of EuroGP'2003, LNCS, vol 2610. Springer, Essex, pp 70–82. https://doi.org/10.1007/3-540-36599-0_7
- Koza JR (1990) Genetic programming: a paradigm for genetically breeding populations of computer programs to solve problems. Technical report, Stanford, CA, USA
- Koza JR (1992) Genetic programming: on the programming of computers by means of natural selection. MIT Press, Cambridge
- Kuber K, Card SW, Mehrotra KG, Mohan CK (2014) Ancestral networks in evolutionary algorithms. In: Proceedings of the companion publication of the 2014 annual conference on genetic and evolutionary computation, pp 115–116
- Langdon WB (1998) Data structures and genetic programming: genetic programming + data structures = automatic programming, volume 1 of genetic programming
- Langdon WB, Banzhaf W (2008) Repeated patterns in genetic programming. *Nat Comput* 7(4):589–613. <https://doi.org/10.1007/s11047-007-9038-8>
- Leung K, Liang Y (2003) Adaptive elitist-population based genetic algorithm for multimodal function optimization, pp 1160–1171. https://doi.org/10.1007/3-540-45105-6_124
- Li JP, Balazs M, Parks G, Clarkson P (2002) A species conserving genetic algorithm for multimodal function optimization. *Evol Comput* 10:207–34. <https://doi.org/10.1162/106365602760234081>
- Li My, Cai Zx, Sun Gy (2004) An adaptive genetic algorithm with diversity-guided mutation and its global convergence property. *J Cent South Univ Technol* 11(3):323–327
- Lichtberger K (2019) Mehrkriterielle evolutionäre optimierung und ihre implementierung als kombination von nsga-ii und alps in heuristiclab. Master's thesis, University of Applied Sciences Upper Austria
- Lobo FG, Lima CF (2007) Adaptive population sizing schemes in genetic algorithms. Springer, Berlin, pp 185–204. https://doi.org/10.1007/978-3-540-69432-8_9
- Luerssen MH (2005) Henotype diversity objectives for graph grammar evolution. In: Recent advances in artificial life. World Scientific, pp 159–170
- Luke S (2000) Two fast tree-creation algorithms for genetic programming. *IEEE Trans Evol Comput* 4(3):274–283
- Mahfoud SW (1996) Niching methods for genetic algorithms. Ph.D. thesis, USA. UMI Order No. GAX95-43663
- Majeed H (2005) A new approach to evaluate GP schema in context. In: Genetic and evolutionary computation conference (GECCO2005) workshop program. ACM Press, Washington, pp 378–381
- Mateiu DD (2019) Open ended symbolic regression with age-layered population structures. Master's thesis, University of Applied Sciences Upper Austria
- Mc Ginley B, Maher J, O'Riordan C, Morgan F (2011) Maintaining healthy population diversity using adaptive crossover, mutation, and selection. *IEEE Trans Evol Comput* 15(5):692–714
- McKay RI, Hoai NX, Whigham PA, Shan Y, O'Neill M (2010) Grammar-based genetic programming: a survey. *Genet Program Evol Mach* 11(3):365–396. <https://doi.org/10.1007/s10710-010-9109-y>
- McPhee NF, Casale MM, Finzel M, Helmuth T, Spector L (2017) Visualizing genetic programming ancestries using graph databases. In: Proceedings of the genetic and evolutionary computation conference companion, GECCO '17. ACM, Berlin, Germany, pp 245–246. <https://doi.org/10.1145/3067695.3075617>
- McPhee NF, Donatucci D, Helmuth T (2016) Using graph databases to explore the dynamics of genetic programming runs. In: Genetic programming theory and practice XIII. Springer, pp 185–201
- McPhee NF, Dramdahl MK, Donatucci D (2015) Impact of crossover bias in genetic programming. In: Silva S, Esparcia-Alcazar AI, Lopez-Ibanez M, Mostaghim S, Timmis J, Zarges C, Correia L, Soule T, Giacobini M, Urbanowicz R, Akimoto Y, Glasmachers T, Fernandez de Vega F, Hoover A, Larranaga P, Soto M, Cotta C, Pereira FB, Handl J, Koutnik J, Gaspar-Cunha A, Trautmann H, Mouret JB, Risi S, Costa E, Schuetze O, Krawiec K, Moraglio A, Miller JF, Wiedera P, Cagnoni S, Merelo J, Hart E, Trujillo L, Kessentini M, Ochoa G, Chicano F, Doerr C (eds) GECCO '15: proceedings of the 2015 annual conference on genetic and evolutionary computation. ACM, Madrid, Spain, pp 1079–1086. <https://doi.org/10.1145/2739480.2754778>
- McPhee NF, Hopper NJ (1999) Analysis of genetic diversity through population history. In: Proceedings of the genetic and evolutionary computation conference, vol 2. Citeseer, pp 1112–1120
- Mengshoel OJ, Goldberg DE (1999) Probabilistic crowding: deterministic crowding with probabilistic replacement. <https://doi.org/10.1184/R1/6710189.v1>
- Mengshoel OJ, Goldberg DE (2008) The crowding approach to niching in genetic algorithms. *Evol Comput* 16(3):315–354. <https://doi.org/10.1162/evco.2008.16.3.315>. (PMID: 18811245)
- Meyer-Nieberg S, Beyer HG (2007) Self-adaptation in evolutionary algorithms. Springer, Berlin, pp 47–75. https://doi.org/10.1007/978-3-540-69432-8_3
- Michalewicz Z (1996) Genetic algorithms + data structures = evolution programs, 3rd edn. Springer, Berlin
- Michalewicz Z, Schmidt M (2007) Parameter control in practice. Springer, Berlin, pp 277–294. https://doi.org/10.1007/978-3-540-69432-8_14
- Misevičius A (2011) Generation of grey patterns using an improved genetic evolutionary algorithm: some new results. *Inf Technol Control* 40(4):330–343
- Neshatian K, Zhang M (2009) Genetic programming for feature subset ranking in binary classification problems. In: European conference on genetic programming. Springer, pp 121–132
- Nguyen QU, Pham TA, Nguyen XH, McDermott J (2016) Subtree semantic geometric crossover for genetic programming. *Genet Program Evol Mach* 17(1):25–53
- Patel S, Clack CD (2007) ALPS evaluation in financial portfolio optimisation. In: 2007 IEEE congress on evolutionary computation, pp 813–819
- Pétrowski A (1996) A clearing procedure as a niching method for genetic algorithms. In: Proceedings of IEEE international conference on evolutionary computation, pp 798–803
- Poli R, Langdon WB (1998) Schema theory for genetic programming with one-point crossover and point mutation. *Evol Comput* 6(3):231–252. <https://doi.org/10.1162/evco.1998.6.3.253>
- Poli R, McPhee NF (2003) General schema theory for genetic programming with subtree-swapping crossover: part I. *Evol Comput* 11(1):53–66
- Poli R, McPhee NF (2009) General schema theory for genetic programming with subtree-swapping crossover: part II. *Evol Comput* 11:169–206
- Poli R, Langdon WB (1997) An experimental analysis of schema creation, propagation and disruption in genetic programming. Technical report CSRP-97-8, University of Birmingham, School of Computer Science. Presented at ICGA-97
- Poli R, McPhee NF (2009) General schema theory for genetic programming with subtree-swapping crossover: part II
- Poli R, Vanneschi L, Langdon WB, McPhee NF (2010) Theoretical results in genetic programming: the next ten years? Genetic programming and evolvable machines 11(3/4):285–320. Progress in Genetic Programming and Evolvable Machines, Tenth Anniversary Issue. <https://doi.org/10.1007/s10710-010-9110-5>

- Rechenberg I (1965) Cybernetic solution path of an experimental problem. Royal Aircraft Establishment Library Translation 1122
- Rosca JP (1995) Towards automatic discovery of building blocks in genetic programming. In: Working notes for the AAAI symposium on genetic programming, vol 445. MIT, Cambridge, MA, USA, AAAI, pp 78–85
- Samuel AL (1959) Some studies in machine learning using the game of checkers. *IBM J Res Dev* 3(3):210–229
- Schmidt M, Lipson H (2011) Age-fitness pareto optimization. In: Genetic programming theory and practice VIII. Springer, pp 129–146
- Shimodaira H (1997) DCGA: a diversity control oriented genetic algorithm. In: Proceedings of the 9th international conference on tools with artificial intelligence, ICTAI '97. IEEE Computer Society, USA, p 367
- Shirkhorshidi AS, Aghabozorgi S, Wah TY (2015) A comparison study on similarity and dissimilarity measures in clustering continuous data. *PLoS ONE* 10(12):144
- Singh G, Deb K (2006) Comparison of multi-modal optimization algorithms based on evolutionary algorithms. In: Proceedings of the 8th annual conference on genetic and evolutionary computation, GECCO '06. Association for Computing Machinery, New York, NY, USA, pp 1305–1312. <https://doi.org/10.1145/1143997.1144200>
- Smart W, Andreae P, Zhang M (2007) Empirical analysis of GP tree-fragments. In: Ebner M, O'Neill M, Ekárt A, Vanneschi L, Esparcia-Alcázar AI (eds) Proceedings of the 10th European conference on genetic programming, lecture notes in computer science, vol 4445. Springer, Valencia, Spain, pp 55–67. https://doi.org/10.1007/978-3-540-71605-1_6
- Smit SK, Eiben AE (2009) Comparing parameter tuning methods for evolutionary algorithms. In: 2009 IEEE congress on evolutionary computation, pp 399–406. <https://doi.org/10.1109/CEC.2009.4982974>
- Smith RE, Forrest S, Perelson AS (1993) Searching for diverse, cooperative populations with genetic algorithms. *Evol Comput* 1(2):127–149. <https://doi.org/10.1162/evco.1993.1.2.127>
- Tackett WA (1995) Mining the genetic program. *IEEE Expert* 10(3):28–38. <https://doi.org/10.1109/64.393140>
- Tate DM, Smith AE (1995) A genetic approach to the quadratic assignment problem. *Comput Oper Res* 22(1):73–83. [https://doi.org/10.1016/0305-0548\(93\)E0020-T](https://doi.org/10.1016/0305-0548(93)E0020-T). (**Genetic Algorithms**)
- Turing AM (1950) Computing machinery and intelligence. *Mind* 49:433–460
- Ursem RK (2002) Diversity-guided evolutionary algorithms. In: International conference on parallel problem solving from nature. Springer, pp 462–471
- Valiente G (2001) An efficient bottom-up distance between trees. In: SPIRE, pp 212–219
- von Neumann J (1966) Theory of self-reproducing automata. *Science* 157(3785):180. <https://doi.org/10.1126/science.157.3785.180>
- White DR, McDermott J, Castelli M, Manzoni L, Goldman W, Kronberger G, Luke S, White DR, Castelli M (2013) Better gp benchmarks: community survey results and proposals. In: Genetic programming and evolvable machines, pp 3–29
- Widera P, Garibaldi JM, Krasnogor N (2010) GP challenge: evolving energy function for protein structure prediction. *Genet Program Evol Mach* 11(1):61–88. <https://doi.org/10.1007/s10710-009-9087-0>
- Wilson GC, Heywood MI (2005) Context-based repeated sequences in linear genetic programming. In: Keijzer M, Tettamanzi A, Collet P, van Hemert JI, Tomassini M (eds) Proceedings of the 8th European conference on genetic programming, lecture notes in computer science, vol 3447. Springer, Lausanne, Switzerland, pp 240–249. <https://doi.org/10.1007/b107383>
- Winkler SM, Affenzeller M, Burlacu B, Kronberger G, Kommenda M, Fleck P (2018) Similarity-based analysis of population dynamics in genetic programming performing symbolic regression. In: Genetic programming theory and practice XIV. Springer, pp 1–17
- Yin X, Gernay N (1993) A fast genetic algorithm with sharing scheme using cluster analysis methods in multimodal function optimization. In: Artificial neural nets and genetic algorithms. Springer, pp 450–457
- Yu E, Suganthan P (2010) Ensemble of niching algorithms. *Inf Sci* 180(15):2815–2833. <https://doi.org/10.1016/j.ins.2010.04.008>
- Zhang K, Statman R, Shasha D (1992) On the editing distance between unordered labeled trees. *Inf Process Lett* 42(3):133–139. [https://doi.org/10.1016/0020-0190\(92\)90136-J](https://doi.org/10.1016/0020-0190(92)90136-J)

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.